TEAM GLITCH

ENGINEERING JOURNAL

WRO FUTURE ENGINEERS SEASON 2026

TEAM: ZEUS WADIA
SHAURYA SULE
REHAAN DHANDHIA

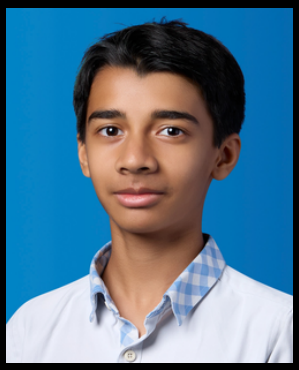
TEAM MEMBERS

Electronics, Testing, Mechanical Design

Zeus Wadia is a Grade 9 student at DAIS with a strong interest in STEM and robotics. Outside robotics, he enjoys playing and following sports.

Vision, Electronics, Mechanical Design

Shaurya Sule is a Grade 9 student passionate about robotics, STEM, and humanities. He also enjoys debating and sports.



Software, Vision & Sensor Integration

Rehaan Dhandhia is a Grade 9 student at DAIS, passionate about programming and robotics. He also enjoys badminton and playing the piano.

Check Out Our Team Links

Our Github Repository

Our Youtube Channel

ROBOT KEY SPECIFICATIONS

Parameter	Value
Dimensions	190 x 120 x 295 mm
Weight	1.35 kg
Drive Type	Rear-wheel differential drive with 12V DC motor
Steering	Servo-actuated parallel steering
Primary Brain	Raspberry Pi 4 Model B (4 GB)
Co-processor	Seeed XIAO ESP32-C3 (IMU + encoder over USB UART)
Vision	HikVision DS-UO2 USB Camera + OpenCV HSV pipeline
Heading	Adafruit BNO085 9-DOF IMU (on XIAO, I2C)
Battery	LiPo 11.1V, 850mAh, 30C
Primary Programming Language	Python
Secondary Programming Language	C++
Max Operating Speed	1.04 m/s

Points Covered: -

LABELLED FRONT VIEW

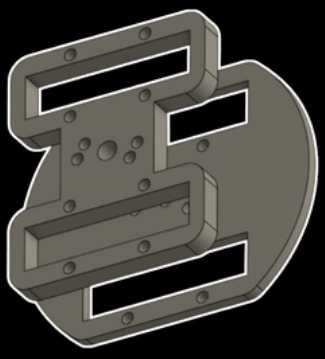
Vikram 453R6
Motor Driver

BNO085 IMU

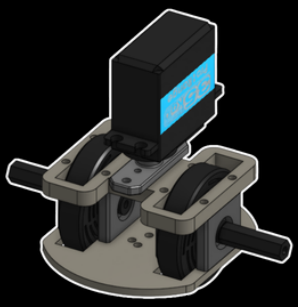
HikVision USB
Webcam

Johnsons 900 RPM
12V DC Motor

35kg-cm Servo



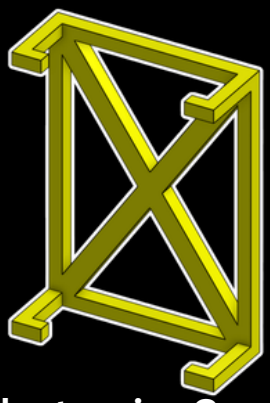
Mild Steel Deadweight



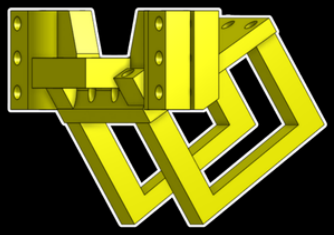
Parallel servo
steering

Points Covered: -

LABELLED BACK VIEW

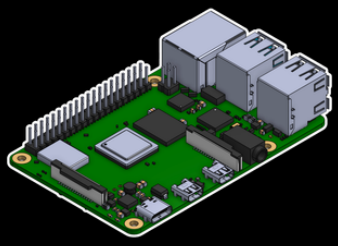


Electronics Cover

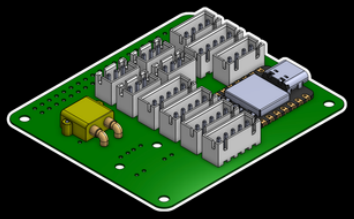


3D printed clamped camera mount

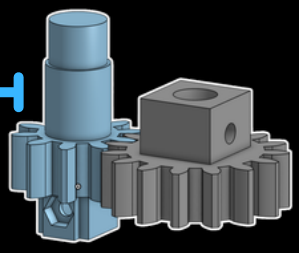
11.1V 850mAh LiPo Battery



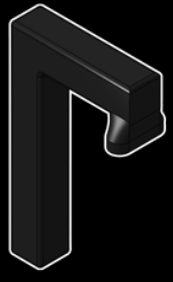
Raspberry Pi 4B



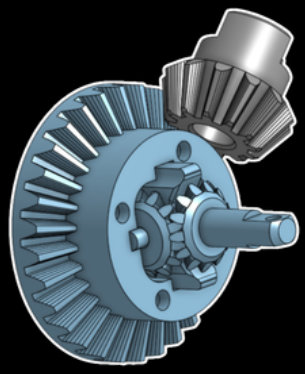
Custom PCB



Secondary Gearbox (Gear Reduction)



Rear Pivot Restrictor

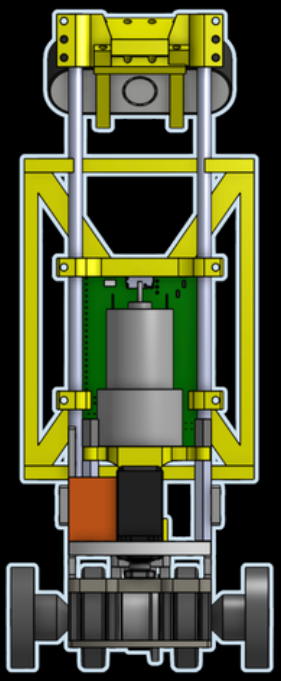


1:3 Metal Differential Gearbox

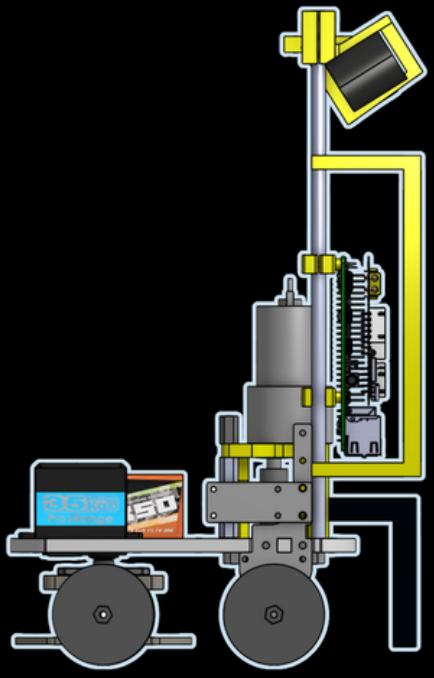
Points Covered: -

ROBOT CAD VIEWS

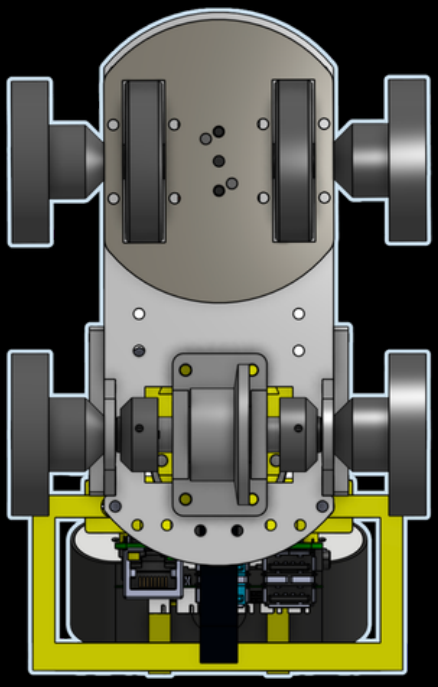
Front View:



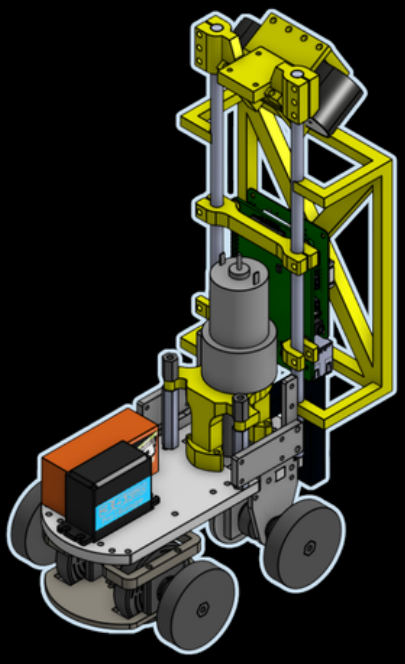
Side View:



Bottom View:



Isometric View:

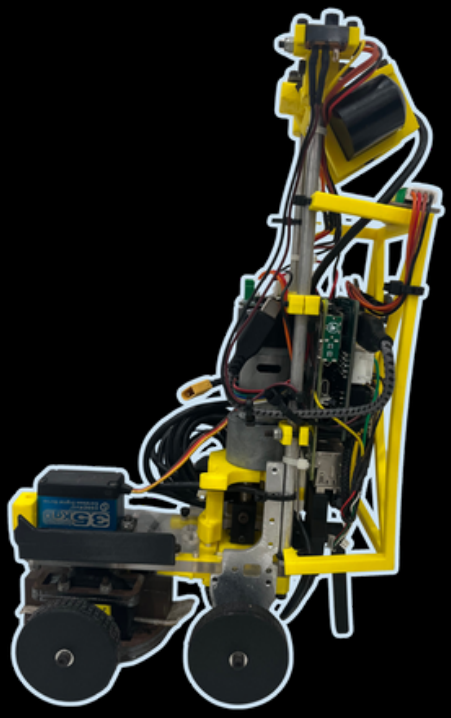


Points Covered: -

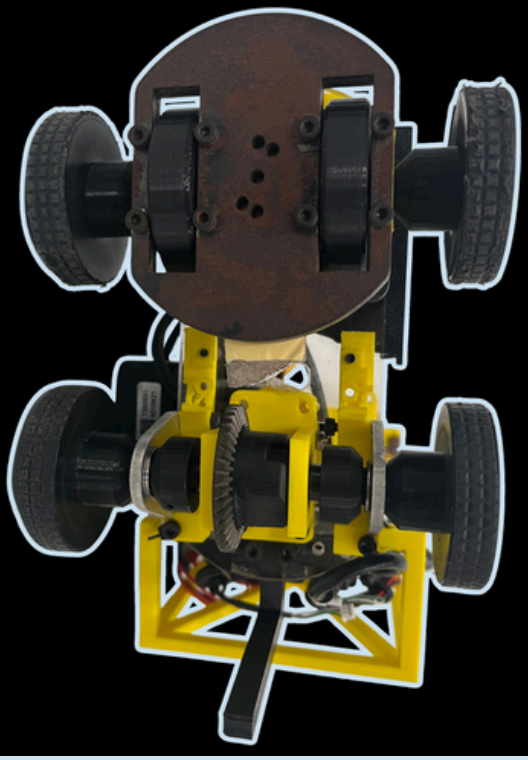
ROBOT REGULAR VIEWS

Front View:

Side View:



Bottom View:



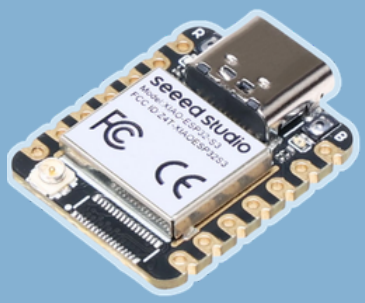
Isometric View:

ONBOARD ELECTRONICS

Raspberry Pi 4B (4 GB)

- Quad-core Cortex A72 processor, 4 GB LPDDR4 RAM, 40 GPIO pins, USB 3.0, 2x Micro-HDMI ports
- Primary microcontroller, Vision compute, sensor control, PID control via PIGPIO.

Seeed XIAO ESP32C3:



- 160 32-bit RISC-V single core (processor), 400 KB SRAM, 11 GPIO I/O pins
- Co-processor, controls BNO085 IMU, external encoder, UART communication with Pi (15200 baud rate).

TF Mini Plus LiDAR

- 0.1-12 meter range, ±5cm accuracy, 3.6° FOV, 5V, UART capability (152000 bps)
- Used to detect distance from wall (on right-side of bot) during parking sequence.

Adafruit BNO085 IMU

- 1.7-3.6V, 9 Degrees of Freedom
- Used to identify heading (used for PID steering).

ONBOARD ELECTRONICS

HikVision DS-UO2 Webcam

- 1920x1080 resolution, 30 FPS, Wide FOV, USB 2.0
- Main source of input - robot is mainly vision based. Gives input to OpenCV HSV Pipeline.

Johnsons 900 RPM DC Motor

- 900 RPM, 1.2 kg · cm rated torque, 800mA rated current
- Used for rear wheel drive via Vikram 453R6 motor driver.

DS3235 35 kg · cm Servo

- 0.18 sec / 60° (at 5V), 23 kg · cm torque (at 5V).
- Controls parallel steering (at 5V).

Vikram-453R6 Motor Driver

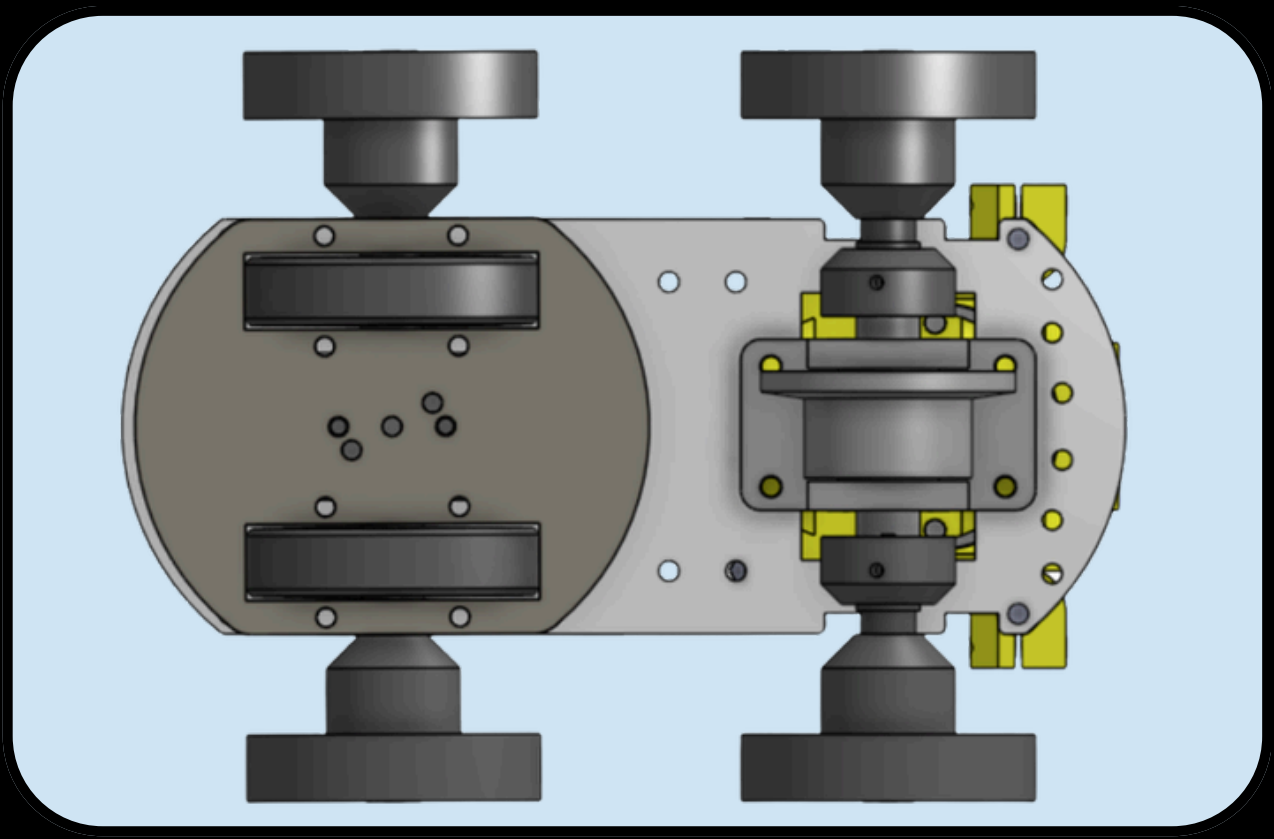
- Single channel H bridge, 6.5-45V DC, 2A continuous output, 3.6A output.
- Motor driver for Johnsons DC Motor, auto-recovery on overcurrent.

Custom PCB

- Custom designed using Altium designer, contains JST 2.54 mm pins and USB ports.
- Simplifies wiring, makes sure no wires are loose

SECTION 1

Mobility & Mechanical Design



Points Covered:

- Chassis design choices
- Steering mechanism justification
- Drive mechanism justification
- Torque & speed reasoning
- Mechanical stability

CHASSIS DESIGN

- **Lightweight & Compact**: We have made the robot chassis as small as possible, which minimizes unnecessary weight and keeps it maneuverable in tight spaces.
- **Rigid & Sturdy**: A firm structure provides stability and securely mounts all subsystems for reliable performance.
- **Agility & Durability**: The balance of strength and low weight enables fast movement, smooth handling, and consistent performance throughout the competition.
- **High Stability:** Even though the camera and motor are at the back, the robot retains stability due to a deadweight at the front and a topple restrictor at the back

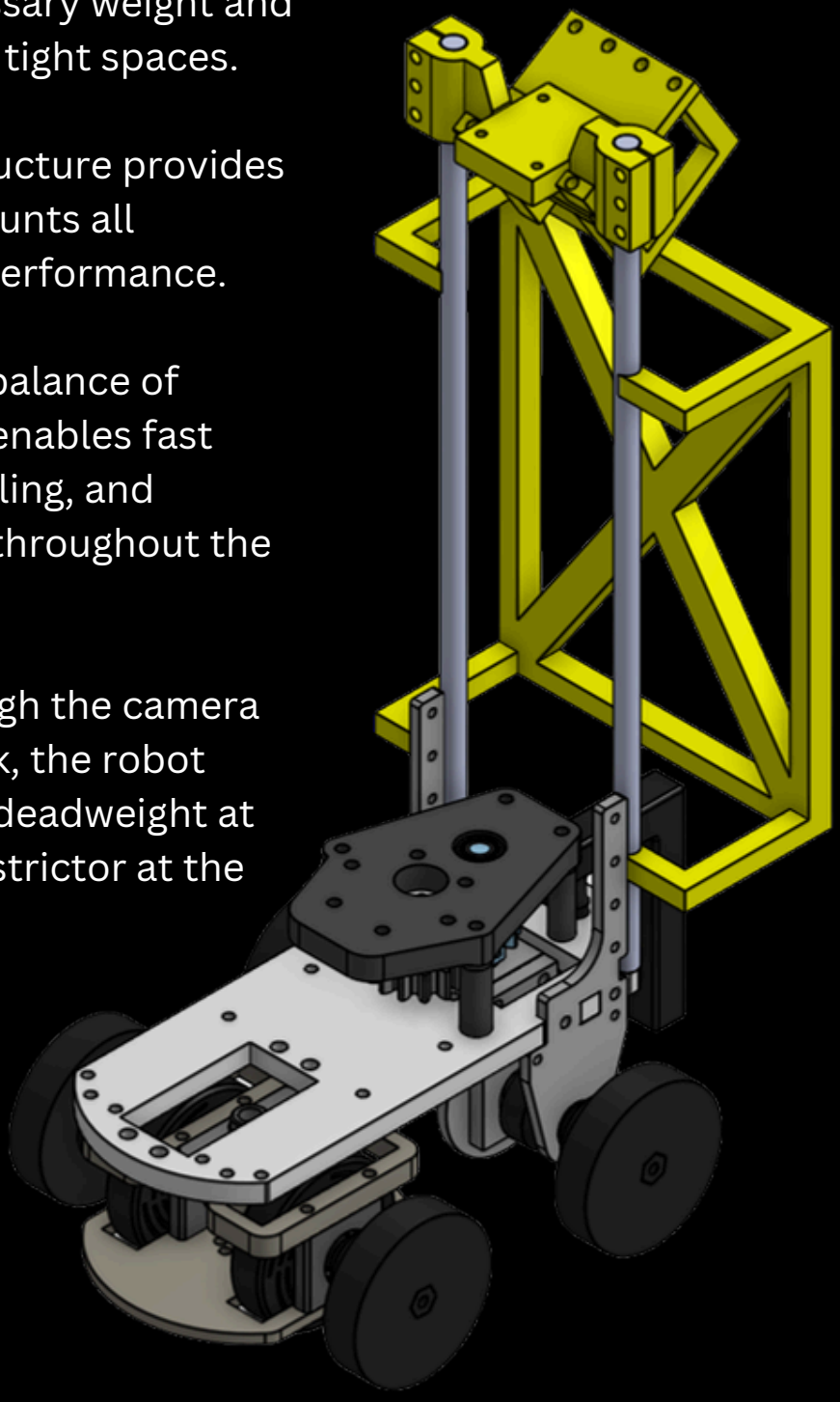


Figure 1.1 - Robot Skeleton

Points Covered: Drive mechanism justification (Section 1), Engineering reasoning for decisions (Section 4)

DRIVE MECHANISM

We use a **Johnson 900 RPM 12V** motor with an off-the-shelf differential gearbox to drive the rear wheels.
A differential gearbox, using a set of 4 perpendicular inner gears, 1 large ring gear, and 1 small bevel gear, allows the 2 output axles to rotate at different speeds. This is particularly important while turning, where each wheel has a different turn radius. Forcing the same angle onto both would result in tyre scrub, slippage, and inefficient power distribution. Figure 1.2 shows the manufacturer's CAD.

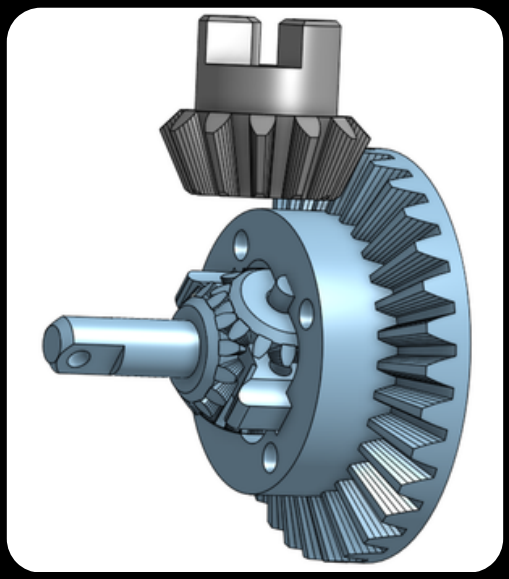


Figure 1.2 - Differential CAD

Why We Chose This:

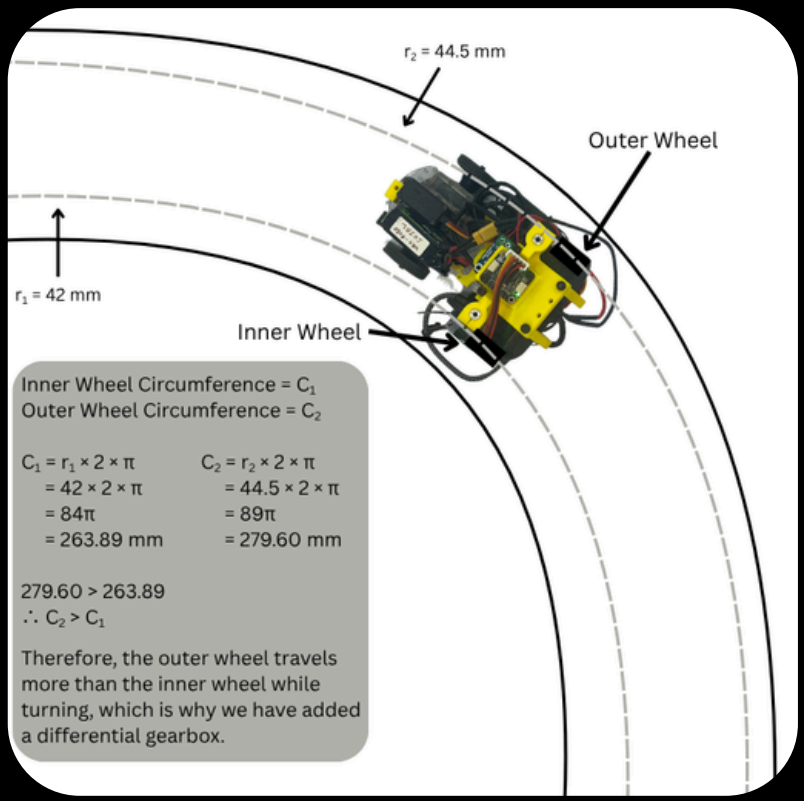


Figure 1.3 - Reason for Differential Gearbox

As shown in Figure 1.3, during a turn, the outer wheel travels a larger radius than the inner wheel. The differential allows the wheels to rotate at different speeds, reducing power loss and wheel slippage and keeping the robot stable.

Points Covered: Steering mechanism justification (Section 1), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4), Iteration & testing cycles (Section 4)

STEERING MECHANISM

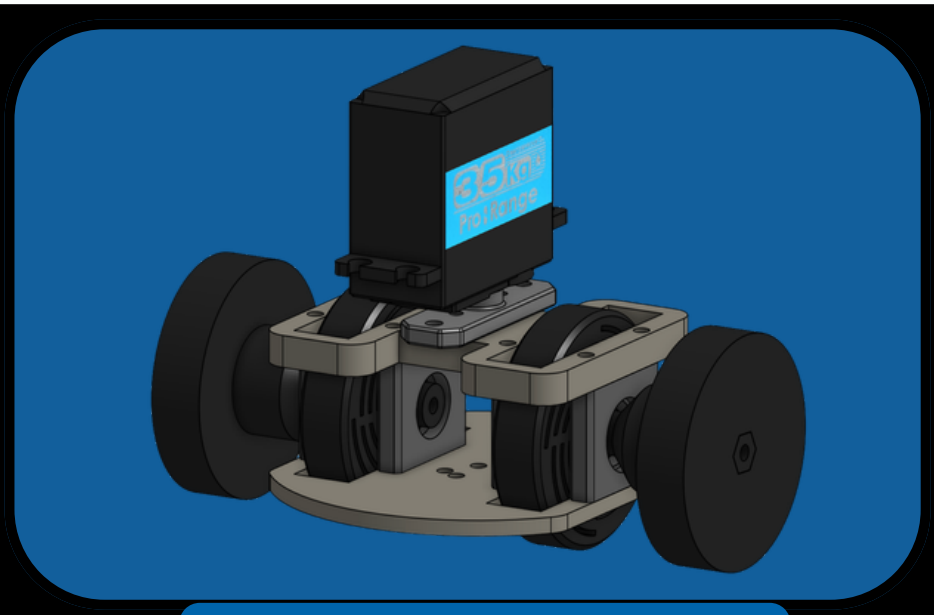


Figure 1.4 - Parallel Steering Linkage

We use a 35 kg servo for steering, connected to a servo horn and a laser-cut steel plate with 3D-printed wheel mounts. The servo provides up to 90° steering for sharp and wide turns.
We tested Ackermann steering (which turns wheels differently according to turn radius), but it was extremely sensitive to loose parts and slipped at higher speeds. This caused unpredictable steering and higher power use, so we chose to use a parallel steering linkage instead.

Acknowledging Trade-Offs:

- **Benefits of Parallel:** Much simpler and more straightforward hardware
- **Drawbacks of Parallel:** Increases weight and tyre scrub at lower speeds. FIXED by adding cat-tongue tape
- **Benefits of Ackermann:** Much more compact and light, aligns wheels to turn radius
- **Drawbacks of Ackermann:** Very sensitive to mechanical play and causes skidding at higher speeds

Points Covered: Steering mechanism justification (Section 1), Torque/speed reasoning (Section 1), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4), Iteration and testing cycles (Section 4)

SERVO CHOICE REASONING...

Analysing Servo Motor Choices

Figure 1.5 - MG996R

Figure 1.6 - GS2010MG

Figure 1.7 - DS3235

Servo Option	Speed (At 5V)	Stall Torque (At 5V)	Verdict
MG996R Servo	0.19 sec / 60°	9.4 kg · cm	Low torque, slow speed. Not ideal choice.
GS2010MG 20kg · cm Servo	0.12 sec / 60°	21 kg · cm	Very high speed, good torque.
DS3235 35kg · cm Servo	0.18 sec / 60°	23 kg · cm	**Current servo choice. Very high torque, slightly slow.**

Acknowledging Trade-Offs:

The DS3235 35kg · cm servo has a lower speed than the GS2010MG Servo (by 0.06 sec / 60°). However its torque (at 5V) is higher than the GS2010MG's by 2 kg · cm. Therefore, overall, we found this change beneficial, reducing risk of the servo failing (due to high load).

TORQUE CALCULATIONS

As shown in Figure 1.8, we calculated our minimum torque requirement for our motor. Estimating a rolling resistance coefficient of 0.03 (everything else was measured), we found that our motor would need around 0.185 kg·cm of torque (considering the 1:2 gear ratio).

Minimum Torque Calculations

Mass of Robot = 1.35 kg

Desired Acceleration = 1.0 m/s²

Net Force = Mass × Acceleration

= 1.35 × 1.0

= **1.35 N**

Gravitational Field Strength = 9.8 N/kg = 9.8 m/s²

Rolling Resistance Coefficient = 0.03

∴ Resistant Force = 9.8 × 0.03 = **0.294 N**

Driving Force = Net Force + Resistant Force

= 1.35 + 0.294

= 1.644 N = 0.168 kgf

∴ Required Driving Force = **0.168 kgf**

Wheel Radius = 2.2 cm

Required Drive Torque = Force × Radius

= 0.168 × 2.2

= **0.3696 kgcm**

Total Gear Ratio = (1.5:1) × (1:3) = 1:2

∴ Required Motor Torque = Torque ÷ 2

= 0.3696 ÷ 2

= **0.185 kg cm**

Figure 1.8 - Torque Calculations

Points Covered: Torque/speed reasoning (Section 1), Drive mechanism justification (Section 1), Current draw reasoning (Section 2), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4), Iteration & testing cycles (Section 4)

DC MOTOR CHOICE REASONING

Motor Option	RPM	Rated Torque (kg · cm)	Rated Current (mA)	Verdict
Cytron IG42E-04K Planetary DC Motor	1400	2.24	**5500**	We considered this motor as it had high torque and high speed. However its rated current of 5500 mA exceeded our motor driver's max current of 2000 mA
Rhino GB37 1000RPM	1000	**0.17**	300	We used this motor in our first bot iteration, but with a torque of only 0.04 kg · cm above our minimum torque requirement (which at that time was 0.13 kg · cm without secondary gearbox), this motor was inconsistent and often failed.
Johnson 300 RPM Grade A	**300**	3.6	800	We considered using this motor due to its high torque and low current requirement, but after calculating the operational speed it would give our bot (0.28 m/s), we realised it would not be fast enough.
Johnson 900 RPM Grade A	900	1.2	800	**The current motor we are using. Has a much better balance of speed and torque**

Points Covered: Torque/speed reasoning (Section 1), Drive mechanism justification (Section 1), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4)

SPEED AND TORQUE REASONING

While Choosing our DC Motor

- While choosing our DC motor, we tried to **prioritise torque** over speed to make our bot more consistent.
- To do this, we use a Johnsons DC motor with 900 RPM and 1.2 kg · cm of torque.

Figure 1.9 - Johnsons DC Motor

Adding a Second Gearbox (For Speed Increase)

- During the National Championship, we realised that our bot was not very fast (max no-load speed of 0.69 m/s).

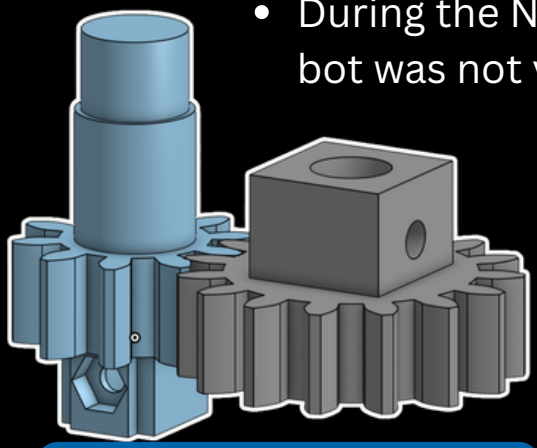


Figure 1.10 - Second Gearbox

- While wishing to increase this, we did not want to change our motor (since it was very consistent) or alter the differential gearbox (since it was store bought).
- We therefore decided to add a second gearbox (connecting the motor to the differential gearbox) with the gear ratio 1.5:1.
- This increased the speed by 50%, reducing torque by 33.3%.

Overview of Speed & Torque Reasoning

- We overall tried to prioritise torque over speed for increased consistency.
- However, observing excessive torque, we attempted to increase speed as much as possible, while still prioritising high torque.

SPEED CALCULATIONS

- The calculations in figure 1.11 helped us estimate the robot's no-load speed (in use) before physical testing.
- By accounting for the motor RPM, gearbox reduction, and wheel circumference, we obtained a **no-load maximum speed of 1.04 m/s**.
- However, we usually run our motor at 80% power, therefore we calculated a **theoretical operational speed of 0.83 m/s.**

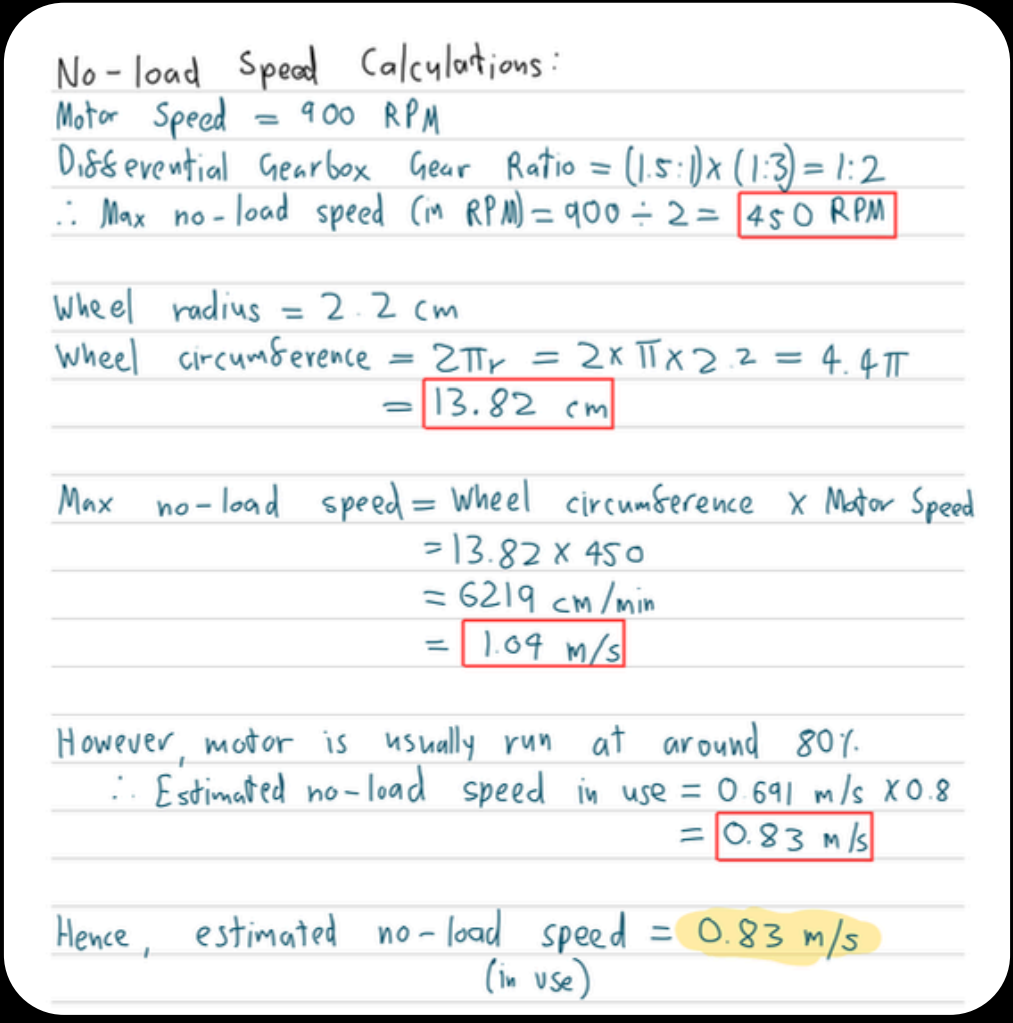


Figure 1.11: Speed Calculations

Points Covered: Mechanical stability (Section 1), Chassis design choices (Section 1), Sensor placement justification (Section 2), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4), Risk identification & mitigation (Section 4)

STABILITY: DEADWEIGHT

Adding a Deadweight

- For our robot's stability, we focused on having a stable robot while allowing it to get the best vantage point for our camera as well.
- We needed our camera to be mounted high, and we realised that creating a large mount with a lot of support across the robot would not only be resource inefficient but also heavier than intended.
- Hence, we chose to leave the camera mount as it was, and a deadweight in the front such that the robot does not topple over.

Acknowledging Trade-offs

- Increased mass of robot by around 100 grams.
- Reduced max speed due to mass increase (~100 grams).

Change in Center of Mass

- Figure 1.12 compares the center of mass of our robot with and without the deadweight
- Note that this diagram is generated by and exported from Onshape based on our robot's CAD (all materials and dimensions set accurately).

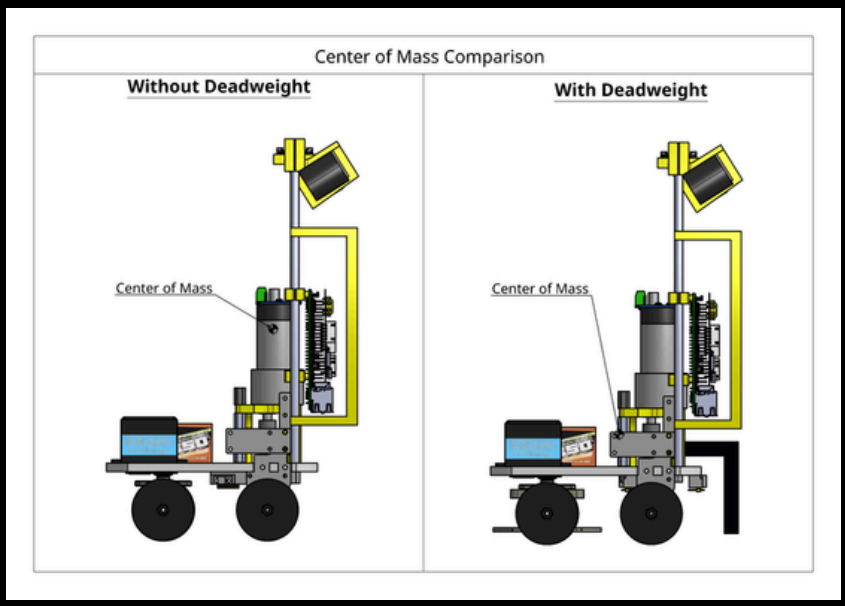


Figure 1.12 - Center of Mass Comparison

Points Covered: Mechanical stability (Section 1), Chassis design choices (Section 1), Engineering reasoning for decisions (Section 4), Constraints & trade-offs (Section 4), Risk identification & mitigation (Section 4)

REAR PIVOT RESTRICTOR

Risk Mitigation:

To decrease risk of our robot toppling, we decided to add a rear pivot restrictor on the rear end of our robot. This ensures that if our robot loses balance falls backward, it does not fully topple over. This improved consistency.

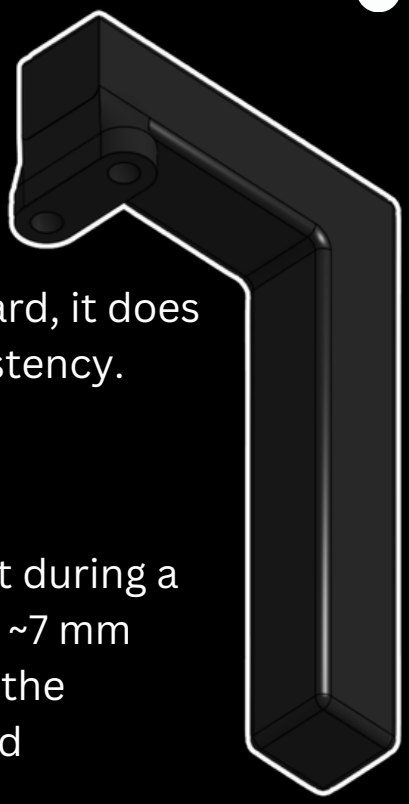


Figure 1.13 - Rear Pivot Restrictor

Not Damaging the Mat

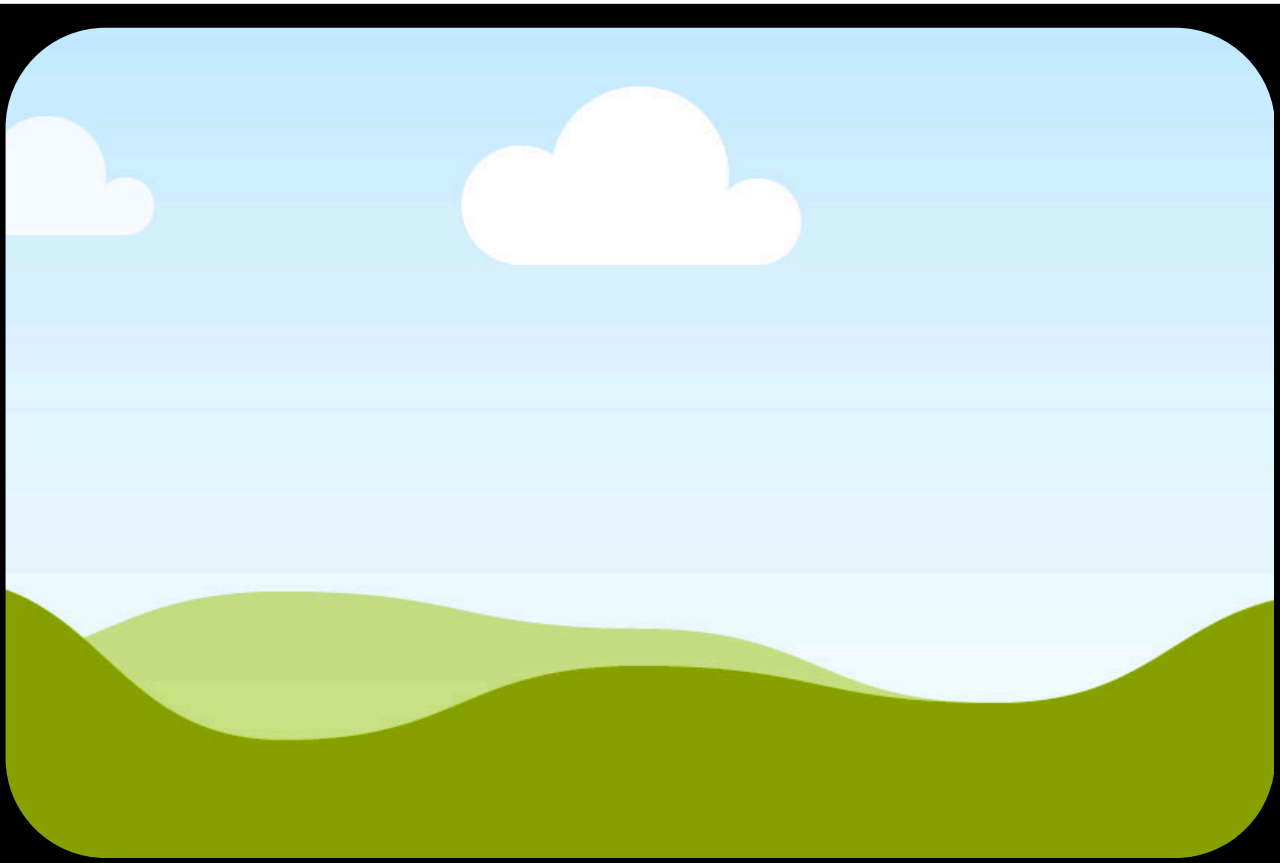
To ensure that this did not damage the mat during a run, we ensured that the pivot restrictor is ~7 mm above the contact between the wheel and the mat, preventing it from dragging across and potentially damaging the mat.

Acknowledging Trade-Offs

- One drawback of using this pivot restrictor is that it moves the center of mass of the robot further towards the rear end of our robot (where our robot was already back heavy).
- However, being made of PLA, it only had a mass of 11.5 grams, making the change in center of mass almost negligible.
- Overall, using the rear pivot restrictor helped safeguard our runs, mitigating risk of our robot toppling over.

SECTION 2

Power & Sensor Architecture



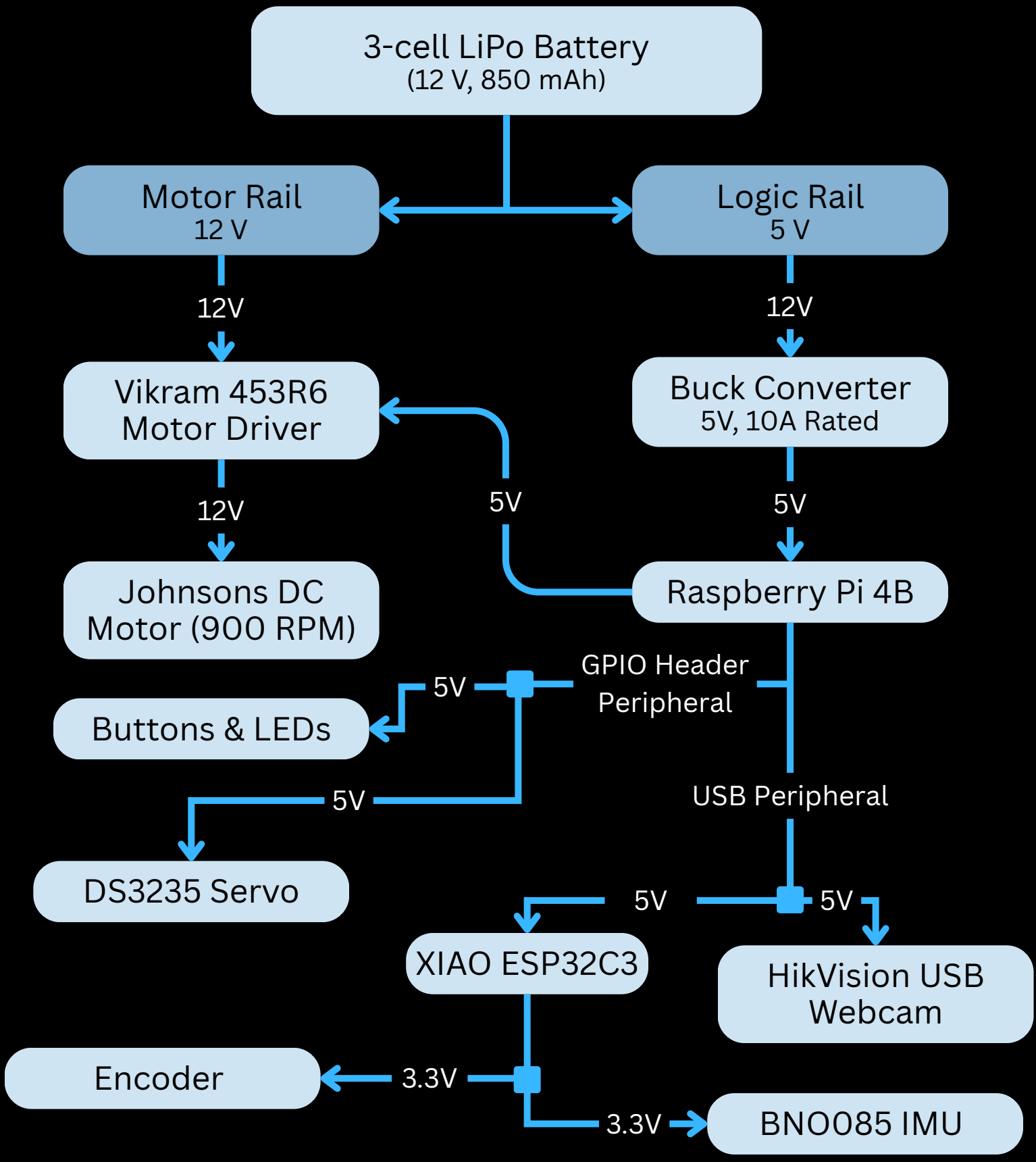
Points Covered:

- Power system architecture
- Current draw reasoning
- Sensor selection justification
- Sensor placement justification
- Calibration
- Wiring diagrams

WIRING DIAGRAM

Figure 11

POWER RAILS


3-cell LiPo Battery
(12 V, 850 mAh)
Motor Rail
12 V
Logic Rail
5 V
12V
12V
Vikram 453R6
Motor Driver
Buck Converter
5V, 10A Rated
12V
5V
5V
Johnsons DC
Motor (900 RPM)
Raspberry Pi 4B
GPIO Header
Peripheral
5V
Buttons & LEDs
5V
USB Peripheral
DS3235 Servo
5V
5V
XIAO ESP32C3
HikVision USB
Webcam
Encoder
3.3V
3.3V
BNO085 IMU

POWER RAILS

Power Rails: Overview

- **12V LiPo Supply:** Our battery powers the system at 12V.
- **Dual-Rail System:** The robot uses separate power rails for the motor, and the other electronics.

- **Motor Rail (12V):** The motor rail is directly powered by the battery at 12 volts. The battery gives 12 volts to the motor driver (Vikram 453R6), which then powers the DC motor directly at 12V.

- **Electronics Rail (5V):** The battery provides 5V to the Raspberry Pi via a buck converter (which regulates the circuit down to 5V).
  - **GPIO Header Peripheral:** Via the Raspberry Pi's GPIO pins, the DS3235 Servo, buttons and LEDs are given 5V.
  - **USB Peripheral:** Via the Raspberry Pi's USB ports, the Hikvision USB Camera and XIAO ESP32C3 are given 5V.
    - **3.3V Sensors:** The XIAO then regulates it down to 3.3V, powering the BNO085 IMU and encoder at 3.3V.

Benefits of Dual-Rail System

- **Noise Isolation:** If on a single rail, electromagnetic interference (electrical noise) can cause sensor glitches and innacuracies in sensor readings. Separating the motor rail and logic rails prevents this.

Points Covered: Power system architecture (Section 2), Engineering reasoning for decisions (Section 4), Risk identification and mitigation (Section 4)

CUSTOM PCB

The Problem

- Originally, our wiring was extremely messy, as we had single jumpers coming out of the Raspberrry Pi, ESP32, and sensors. It also had large risks.
- It was also very loose and prone to frequent disconnections if the robot hit a wall or obstacle, and some wires would come out even when it was idle
- Rewiring or changing any modules was extremely impractical as you had to pick out exactly the right wires, without messing with any others

Our Solution

- We designed a custom PCB using Altium designer.
- On one side of our PCB, we have 2 rows of female pin headers, connected directly to the Raspberry Pi's GPIO pins.
- For power input (from the battery), we use an XT30 connector.
- To connect to most electronics (sensors, servo, motor driver), we use JST connectors which lock the wires, securing the connections.
- The XIAO ESP32C3 is attached to the the PCB through female pin headers.
- This PCB is then screwed directly onto 3D printed mounts on the rods.
- Overall, using this PCB helps us secure and organise our wiring, mitigating risks.

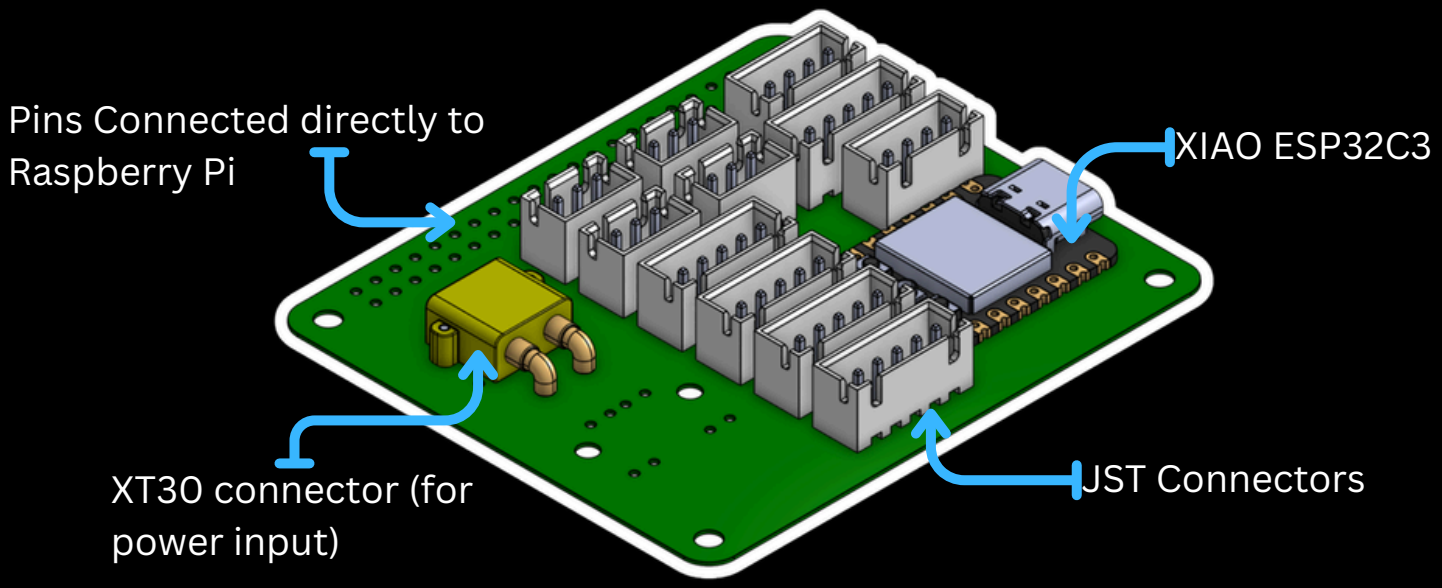


Figure 2. - PCB CAD + Labels

Points Covered: Power system architecture (Section 2), Current draw reasoning (Section 2)

POWER BUDGET ESTIMATE

- This is our power budget estimate for the entire robot, including sensors, actuators, and processors
- The highest consumer is the motor, with 12V 800mAh (9.6A) required while running
- Overall, the total power required is 29 watts, of 40mAh at 12V. Our battery - 12V 850mAh - can supply a sustainable 12V to the robot for at least 5 consecutive runs

Component	Voltage	Expected Current Draw	Expected Power
Johnson 900RPM DC Motor	12V	800 mA running (2400 mA stall)	9.6W (28.8W Stall)
Raspberry Pi 4B	5V	1200 mA	6.0W
XIAO ESP32C3	5V	200 mA	1.0W
DS3235 Servo	5V	1000 mA (1900 mA stall)	5.0W (9.5W Stall)
External Quadrature Encoder	3.3V	3 mA	0.001W
BNO085 IMU	3.3V	7.5 mA	0.025W
Total (Normal)	N/A	~3.2105 A	~21.63W

BATTERY RUNTIME CALCULATIONS

- Figure **2.X** shows our calculations for our battery's estimated runtime.
- As shown in figure **2.X**, our battery can run for ~21 minutes.
- Note that the section labelled "Battery Specifications" gives specifications including voltage (nominal and full charge), capacity, and C-rating.
- Some specifications (like full charge voltage and C-rating) are not required for battery runtime calculations, but we have still provided them.

Battery Specifications:

Nominal Voltage = 11.1 V
Full charge Voltage = 12.6 V
Capacity = 850 mAh
C-rating (Discharge Rating) = 30C

Energy Capacity = Nominal Voltage × Capacity
= 11.1 V × 0.85 Ah
= 9.435 Wh

Discharge Limit = Capacity × C-rating
= 0.85 Ah × 30C
= 25.5 A

Battery Runtime Calculations:

Expected Power Draw = 21.63 W

$$\text{Max Expected Runtime} = \frac{\text{Energy Capacity}}{\text{Power drawn}} = \frac{9.435\ \text{Wh}}{21.63\ \text{W}} = 0.436\ \text{h}$$

= 26.2 minutes

Estimated Efficiency = 0.8 = 80%

Estimated Runtime = Efficiency × Max Runtime
= 26.2 min × 0.8
= 20.96 = 21 min

∴ Estimated Runtime = 21 minutes

Figure 2.x - Battery Runtime Calculations

Points Covered: Sensor selection justification (Section 2), Engineering reasoning for decisions (Section 4), Constraints & Trade-offs (Section 4), Iteration and testing cycles (Section 4)

CAMERA CHOICE

Pixy2 Camera vs Hikvision Webcam

v1 - Pixy2

- The Pixy2 AI Camera has its own onboard processor that extracts colour coordinates and sends them to the processor.
- It had a high 60 FPS frame rate that allowed the robot to react quickly.
- It was a great starting point as it was small, lightweight, and featured easy built-in color detection.
- However, its sow resolution and high sensitivity to shadows/glare made color detection unreliable under different types of lighting, and calibration was extremely difficult.
- It was also not compatible with OpenCV

Figure 2.X - Pixy2

v2 - Hikvision Webcam

- It had a much higher resolution (1080p) which vastly improved image clarity.
- It had a much wider field of view, which allows it to capture a larger area of the track to spot obstacles much earlier.
- It is fully compatible with OpenCV, which lets it give consistent color and adjustable exposure - made obstacle detection highly reliable anywhere.
- We could also program a fast calibration script with OpenCV, allowing the robot to quickly adapt to the field at competitions.
- However, it was much heavier than the Pixy2

Figure 2.X - Hikvision Webcam

Points Covered: Sensor selection justification (Section 2), Engineering reasoning for decisions (Section 4), Constraints & Trade-offs (Section 4), Iteration and testing cycles (Section 4)

CAMERA CHOICE

Pixy2 Camera vs Hikvision Webcam

Feature	Pixy2	HikVision
Resolution	720 P	1080 P
FPS	60	30
FoV (degrees)	60	86
Weight	10g	50g
Lighting adaptability	Low	High
Detection precision	Colour blobs only	Contours, shading

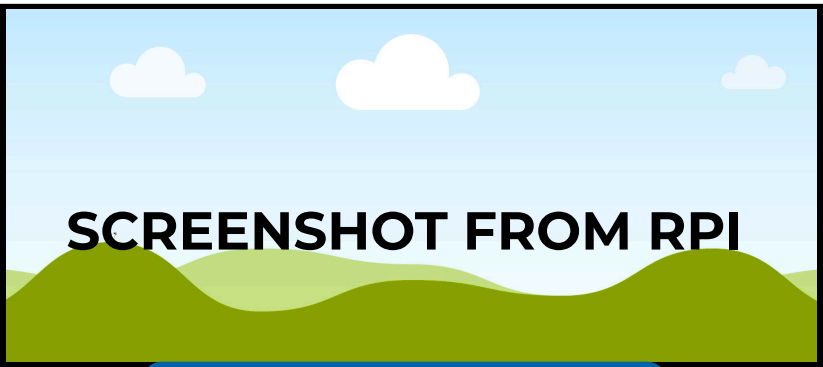


Figure 2.X - RasPi Camera View

IMU CHOICE

MPU6050 vs Acrome BNO055 vs Adafruit BNO085

Our Goals while Choosing an IMU

- **Size:** We wanted our IMU to be compact and able to fit in a small slot (30mm X 25mm) on top of our camera mount.
- **Prioritising Gyroscope:** We wanted our IMU to have a very accurate gyroscope, as we are only using the gyroscope.

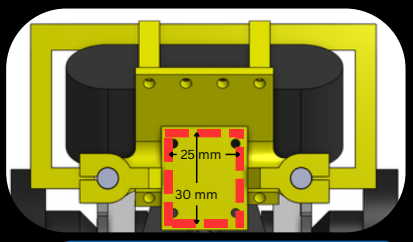


Figure 2.X - IMU Slot

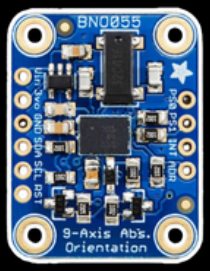


Figure 2.X - MPU6050

Figure 2.X - BNO055

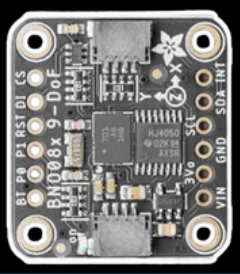


Figure 2.X - BNO085

IMU Option	Gyro Sensitivity Error (Datasheet)	Gyro Sensitivity Error (Testing)	Size - L x B (mm)	Verdict
MPU6050	±3%	±3.78% (90-86.6)/90	21 x 16	Too inaccurate
BNO055	±1%	±1.67% (90-88.5)/90	40 x 20	Too large
BNO085	Not Specified	±0.89% (90-89.2)/90	25.6 x 22.7	**High accuracy and compact size**

Note: Error calculated while testing was calculated over 25 trials (taken average), (to 3 s.f). For 90° rotation, IMU was mounted on DS3235 Servo.

Points Covered: Sensor placement justification (Section 2), Engineering reasoning for decisions (Section 4), Risk identification and mitigation (Section 4)

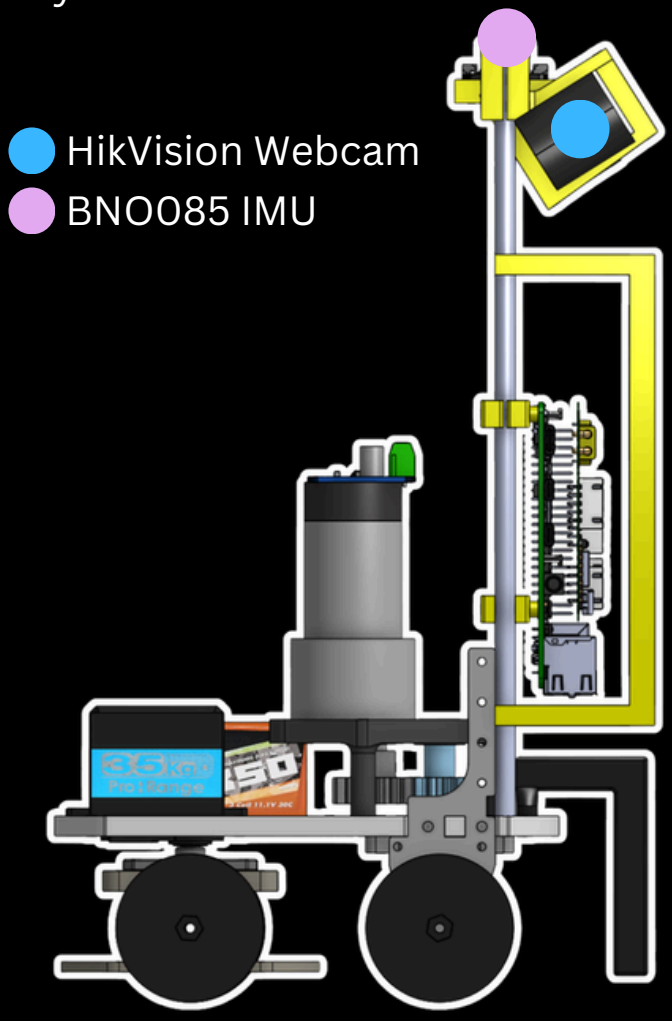
SENSOR PLACEMENT RATIONALE

HikVision Webcam

Mounted at a downward angle right at the top of the robot (lens at 285mm) for a wider detection range. Having such a high mount minimizes dead spots near the robot while still having a large, long field of view. We also set the exposure manually to reduce flickering due to the height.

Adafruit BNO085 IMU

Mounted above the camera, just below the permitted height limit. Since the motor uses magnetic force, it interferes with the IMU's accelerometer. Therefore, we have placed it as far from the motor as possible to minimize magnetic interference, and the shaft holding it is doesn't touch the motor, which minimizes vibratory interference.



COLOUR CALIBRATION STRATEGY

Stage 1: Record

- We created a python script to record everything in our camera's field of view for a set period of time.
- To get a proper recording for calibration, we would manually move the robot, capturing all the game elements.
- LEDs on the back of the robot show us whether the robot is recording or not (red LED shows not recording, green LED shows recording in progress).

Stage 2: Calibrate

- We created a Matlab script allowing us to import a video, navigate to any timestamp, and calibrate any colour by clicking on a specific frame with that colour, and then alter its HSV values using sliders.
- However, the Matlab script was heavy to run, and sometimes faced lots of lag on our laptop.
- Therefore, to mitigate risk of failed calibration during the competition, we decided to also make a Python script run on our Raspberry Pi for the same purpose (which directly pulls the latest video from the "videos" folder).

Stage 3: Verify Calibration

- After calibrating, we can play the entire video.
- This will overlay the areas which match each colour.
- This allows us to check if the OpenCV is detecting the blocks as their respective colours consistently.

SECTION 3

Software Architecture & Obstacle Strategy

```
    """pwm.write(red_led, 0)
    pwm.write(green_led, 1)"""
elif red_b.value and (not green_b.value and time.time() - g_time > 0.01) and not lap_finish:
    r_time = time.time()
    g_time = 0
    centroid = centr_x_red.value
    print("default red following")
    if obstacle_state == 1:
        print(f"center red...")
        if orange_flag:
            if wall_right.value > 0 or outer_wall_right.value > 0:
                err = 340 - centroid
            else:
                err = 300 - centroid
        else:
            err = 250 - centroid
    elif obstacle_state == 2:
        print(f"right red...")
        if orange_flag:
            if wall_right.value > 0 or outer_wall_right.value > 0:
                err = 140 - centroid #140
            else:
                err = 110 - centroid #120
        else:
            err = 100 - centroid
        # err = 120 - centroid

    """pwm.write(red_led, 1)
```

Points Covered:

- Code modularity
- State machines / flow logic
- Lane following & obstacle obedience strategy
- Explanation of algorithms
- Code documentation

Points Covered: - Code modularity (Section 3), Flow logic (Section 3)

SOFTWARE OVERVIEW

- Our programming is split into 2 languages - Python and C++
- Python is the main programming language used, to code the Raspberry Pi. This is the primary software, as it handles drive, steering, and vision, and takes input from the ESP32 for encoder and heading.
- We use C++ to code the ESP32-C3, which takes input from the IMU and encoder.
- The python code on the Pi uses multiprocessing on 3 of the CPU cores.
- All code is available on our Github repository.

Raspberry Pi Process Architecture

- We use multiprocessing on the Pi, which allows us to run our 3 processes - drive, vision, and encoder/IMU - simultaneously
- This decreases the loop time, makes the robot more versatile and makes the code much more modular and easier to maintain

Main Process
Spawns all processes and exits

CameraProcess

- OpenCV HSV pipeline
- Uses the vision_pipeline code for input
- Controls the camera, detects obstacles, walls, lines within the ROIs, and saves the data in multiprocessing variables

DriveProcess

- Main decision-making loop
- Controls the servo & DC motor, takes input from all processes, and contains the main logic
- Decides obstacle avoidance, parking, and PID steering based on IMU, encoder, and camera data

IMUandEncoder

- Communication with ESP32-C3 over USB (UART)
- Sends basic commands and recieves raw IMU and encoder data
- Formats and stores the data in multiprocessing variables for the DriveProcess to use.

Points Covered: - State machines (Section 3), Code modularity (Section 3), Flow logic (Section 3)

OPEN CHALLENGE STATE MACHINE

- Our open challenge code operates using 3 main principles - wall PID, IMU reading PID, and turn logic.
- It uses only 2 sensors - the camera and the IMU's gyroscope.

State Machine

Initalization
Runs all 3 processes
Sets turn counter, IMU, encoder to 0

1st Lane
Sets target heading to 0, runs on IMU PID (difference from 0 is error) until line detected

Direction Decision
Once Line ROI is triggered, the robot sets 'orange_flag' and 'blue_flag' to true/false depending on the colour

While counter <12

Turn
Based on the colour flag, the robot adds 90 to its target heading (mod 360), and takes the turn once the wall ahead enters the 'close_black' region. 1 is added to the turn counter

Straight Lane
The robot runs straight until the line is detected, using IMU correction from target and steering to the middle of the lane if side walls are detected

Last Straight
After the turn, the robot runs straight using only IMU and walls, and stops after 1.4 seconds in the middle of the lane

Points Covered: - State machines (Section 3), Code modularity (Section 3), Flow logic (Section 3)

OPEN CODE EXPLANATION...

- As seen on the previous page, the open challenge operates purely on vision and IMU input, without requiring any other sensor data.
- The primary input source is the HikVision webcam, which uses custom OpenCV Regions of Interest (ROIs) to detect the blue/orange lines, front walls, and side walls.
- The IMU data is constantly sent to the Pi, and is used for heading correction and turns. We use PID, with the lane angle setpoint as the target and deviation from that as the error, to keep the robot from crashing into the walls
- A turn counter tracks each turn, and after 12 turns, the robot uses the ‘far_wall’ ROI to detect the final wall and stop precisely. This proved more reliable than encoder counts or distance sensors.

PD STEERING CONTROL (CORRECTANGLE)

We use a PD controller for our steering mechanism. The integral term *ki* is set to 0 in the obstacle challenge to avoid windup during block avoidance transitions - a lesson learned after the robot over-corrected and hit a wall when *ki* was non-zero.

```
### CORE PID
error_gyro = heading - setPoint_gyro
if error_gyro > 180: #angle wrap around
    error_gyro -= 360
corr = error_gyro

pTerm = kp * error_gyro * multiplier
dTerm = kd * ((error_gyro - prevErrorGyro) / dt)
totalErrorGyro += error_gyro
iTerm = ki * totalErrorGyro
correction = pTerm + iTerm + dTerm
servo.setAngle(95-correction) #95 is the mechanical centere
```

The multiplier parameter scales aggressiveness: 1.0 for normal straight-line heading hold, 1.5 for block-tracking and most driving situations, 3.0 for tight parking turns. This avoids having multiple PID instances for the same physical task. *correctReverseAngle* mirrors the same structure but applies correction in the opposite servo direction (95 + correction) for reverse maneuvers.

OBSTACLE AVOIDANCE - CENTROID PD CONTROL

Our obstacle avoidance uses a 2-stage system: following and avoiding. The robot detects the obstacle's centroid, position, and area, then uses a PD controller to steer towards it. When it is close enough, it checks the colour and turns in the correct direction, then returns to Stage 1 after passing the block.

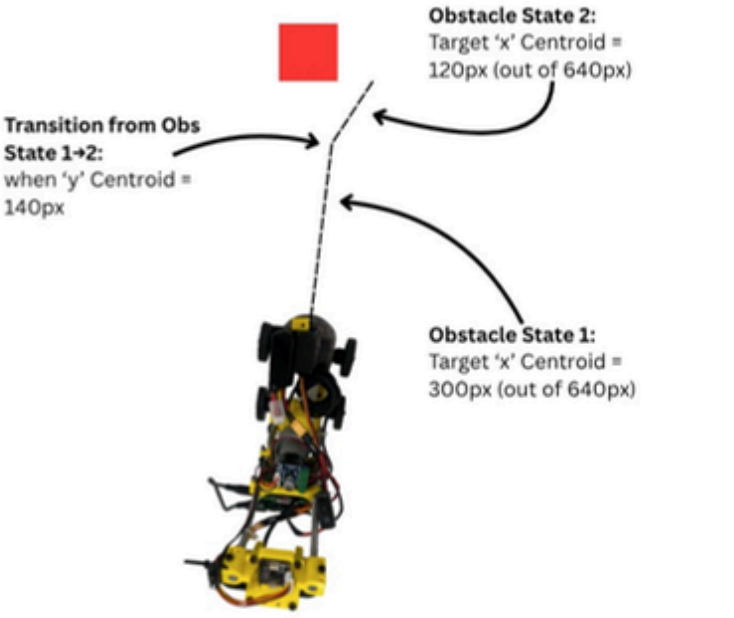
```
if green_b.value and (not red_b.value and time.time() - r_time > 0.01) and not lap_finish: # if green block is deteced
    g_time = time.time()
    r_time = 0
    centroid = centr_x.value
    print("default green following")
    if obstacle_state == 1: #green block is far
        print(f"center greeen...")
        err = 370 - centroid #steer till green is roughly x-axis center of frame (640px frame)
    elif obstacle_state == 2: #green block is near
        print(f"left greeen...")
        err = 520 - centroid #steer left of the green block

elif red_b.value and (not green_b.value and time.time() - g_time > 0.01) and not lap_finish: #if red block is detected
    r_time = time.time()
    g_time = 0
    centroid = centr_x_red.value
    print("default red following")
    if obstacle_state == 1: # red block is far
        print(f"center red...")
        err = 270 - centroid #steer till red is roughly x-axis centre of frame (640px frame)
    elif obstacle_state == 2: #red block is near
        print(f"right red...")
        err = 120 - centroid #steer right of the red block
```

We use a regular PD controller for changing the servo angle based on the offset (error) of the block to the robot. It clamps at +/- 35 degrees as that is the physical limit for the servo in this position

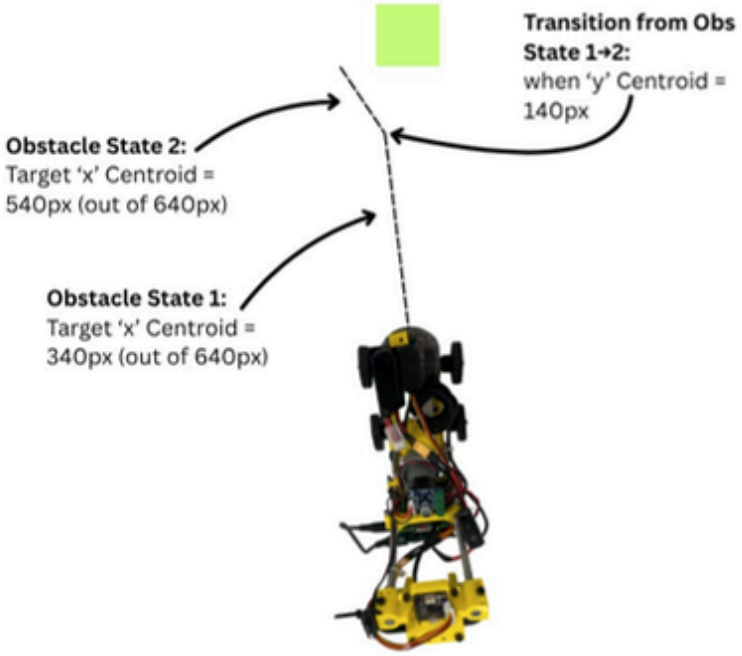
```
servo_angle = err * kp_s + (((err - prev_err) / dt_main) * kd_s)
servo_angle = max(-35, min(35, servo_angle))
prev_err = err
```

OBSTACLE AVOIDANCE LOGIC

Red Obstacle Avoidance



Green Obstacle Avoidance



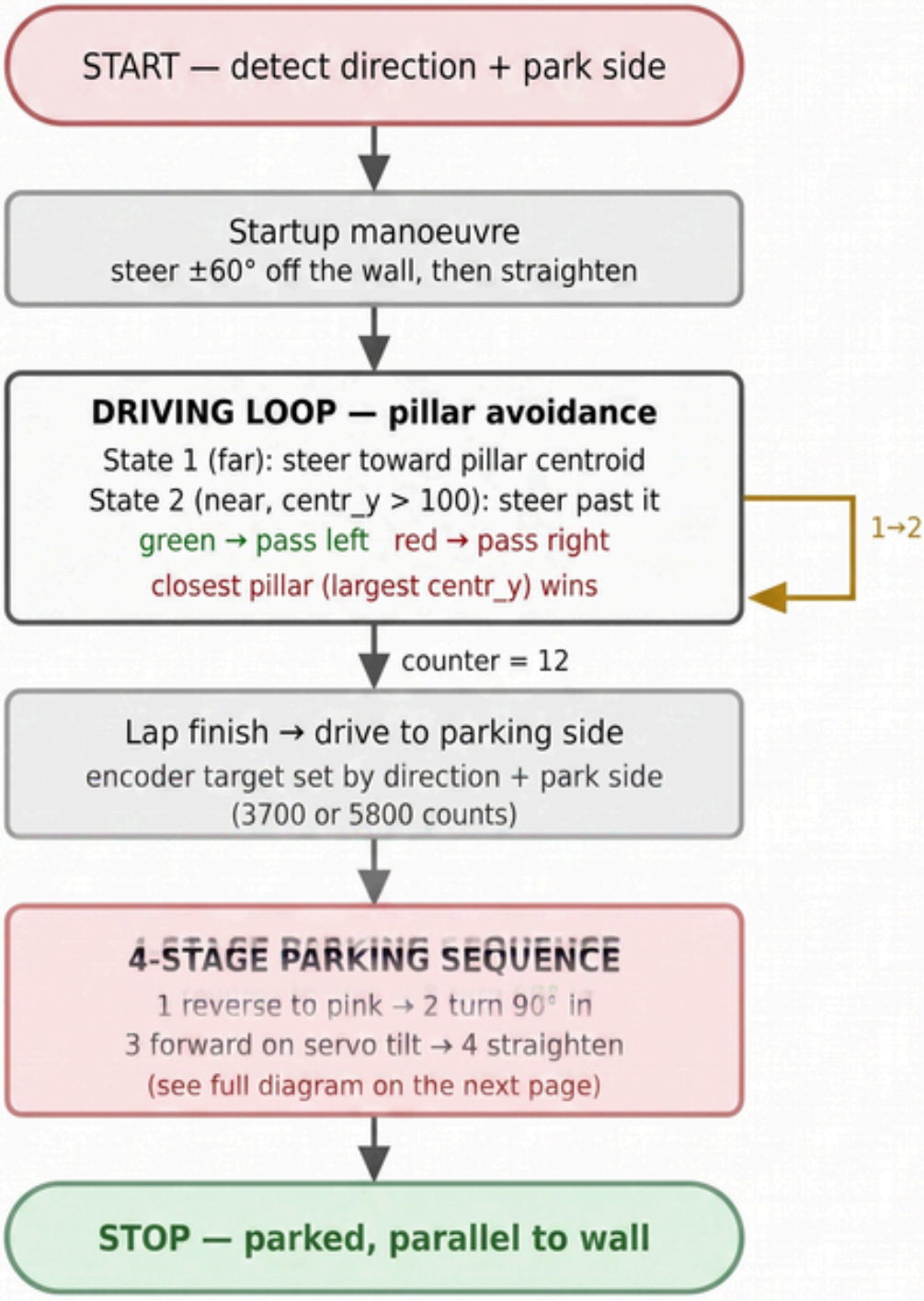
We developed a two-state obstacle avoidance algorithm using HSV colour detection and centroid tracking. The robot identifies red and green obstacles and adjusts its steering based on their position within the camera frame.

State 1 - Initial Avoidance: The robot targets an X-centroid of 300px for red obstacles and 340px for green obstacles, initiating a gradual steering response.

State 2 - Complete Avoidance: Once the obstacle's Y-centroid reaches 140px, the target X-centroid shifts to 120px for red and 540px for green, allowing the robot to steer around the obstacle.

This approach enables smooth steering transitions and controlled obstacle avoidance.

OBSTACLE CHALLENGE



The two-state pillar logic exists because a single "steer away" rule made us clip pillars we were still approaching head-on.

OBSTACLE CHALLENGE PROGRAMMING STRATEGY

FULL STATE MACHINE

```
[STARTUP]
    |
    ├── Init CameraProcess (P)  — OpenCV HSV pipeline
    ├── Init IMUandEncoder (E)  — XIAO UART reader (BNO085 + encoder)
    ├── Init TOFProcess (T)     — VL53L0X ×4
    └── Start DriveProcess (S)  — main decision loop
          |
          ▼
[DIRECTION + PARKING SIDE DETECTION]
    |  (outer_wall_left > 0 AND NOT outer_wall_right) OR (tf_l < tf_r)
    |       → orange_flag = True (parking lot is on the right, CW)
    |  (outer_wall_right > 0 AND NOT outer_wall_left) OR (tf_r < tf_l)
    |       → blue_flag = True (parking lot is on the left, CCW)
    ▼
[STARTUP MANEUVER — inParkingatStart]
    |  Robot steers ±60° away from wall then straightens
    |  (orange: correctAngle(heading + 60), blue: correctAngle(heading - 60))
    ▼
[DRIVING LOOP — counter < last_counter (default 12 = 3 laps × 4 turns)]
    |
    ├── Pillar detected?
    |       Green → err = 350 - centr_x (state 1) / 560 - centr_x (state 2)
    |       Red   → err = 290 - centr_x_red (state 1) / 80 - centr_x_red (state 2)
    |       None  → wall_left/right centroid → err = ±15 nudge
    |       Pink + continue_parking → err toward centr_x_pink setpoint
    |
    ├── Centroid PD: servo_angle = err * kp_s + (err_diff / dt) * kd_s
    |       obstacle_state 1→2 when centr_y > 100 (block close)
    |       obstacle_state 2→1 when block clears or pink/close_wall nearby
    |
    ├── Turn trigger: orange_l (CW) or blue_l (CCW) floor line detected
    |       → trigger = True, counter_flag = False
    |       → counter++ when trigger fires
    |       → heading_angle updated via update_heading(counter, ...)
    |       → trigger resets after 2s timeout
    |
    └── No block + outer walls visible: correctAngle(heading ± 20) wall bias
```

OBSTACLE CHALLENGE PROGRAMMING STRATEGY

FULL STATE MACHINE

```
[LAP FINISH — counter == last_counter]
    |
    ├── Drive to closest heading setpoint, wait for close_wall to appear
    ├── Reverse to align (correctReverseAngle), then resume forward
    ├── Compute target encoder count from parking side + direction:
    |       orange + parking_right: counts + 5800
    |       orange + parking_left:  counts + 3700
    |       blue   + parking_right: counts + 3700
    |       blue   + parking_left:  counts + 5800
    |       (finish_thresh: 1400 or 1900 depending on config)
    ├── Drive forward until counts >= target_count
    └── Stop → lap_finish = True

[PRE-PARKING — lap_finish, NOT continue_parking]
    |  orange_flag: heading_angle += 90 (turn to face slot)
    |  blue_flag:   heading_angle -= 95 (turn to face slot)
    |  Reverse to align (correctReverseAngle), drive until park_wall > 4500 area
    |  then forward until park_wall < 4500, stop → parking_STATE = 2
    |  Adjust heading ±95°, reverse until corr < 5° → continue_parking = True

[CONTINUE PARKING — continue_parking, NOT parking_flag]
    |  Drive forward at power=50
    |  If last_wall > 1350 AND heading aligned AND no pink → parking_flag = True
    |  Steer toward pink centroid (centr_x_pink) if visible, else correctAngle

[MAIN PARKING — parking_flag]
    |
    ├── [STATE 1] Reverse 100 encoder counts straight
    |
    ├── [STATE 2] heading_angle ± 90° (into slot)
    |       Reverse until corr < 10°, then reverse 1.5s more
    |
    ├── [STATE 3] Forward 0.55s: first 0.2s straight, then steer into slot
    |       (parking_right → servo 70°, parking_left → servo 110°)
    |
    └── [STATE 4] heading_angle - 90° (straighten)
            Reverse until corr < 3°
            Drive forward until centr_y_pink > 235 (fully inside lot)
            → Motor stop, servo center, sys.exit(0)
```

VISION SYSTEM - OPENCV HSV PIPELINE

The *CameraProcess* runs vision_pipeline_new.py - a pure OpenCV HSV colour-segmentation pipeline with no ML accelerator required. It detects the following colour classes across named spatial zones:

Colour	Zone(s)	**Role**
Red	main, close	Red traffic sign pillar
Green	main, close	Green traffic sign pillar
Magenta	main, close	Parking lot boundary marker
Orange	line, line2	Floor line - CW lap direction
Blue	line, line2	Floor line - CCW lap direction
Black	wall_inner_left/right, wall_left/right, close_black, parking_wall, last_wall	Inner/outer wall and parking zone boundaries

Why HSV over ML?

Last season we used a Google Coral Edge TPU with a quantized TFLite model. This season we moved to an **HSV pipeline** for simpler deployment (no TPU driver stack), faster iteration on colour thresholds, and elimination of the TPU as a failure point. To handle lighting variation we run the pipeline in both HSV and LAB colour spaces (USE_LAB flag) and tune thresholds per colour class.

VISION SYSTEM - OPENCV HSV PIPELINE

Detection logic: Each frame's detections are filtered by zone and sorted by centroid-Y (largest Y = closest to robot). If red and green pillars are both visible simultaneously, the one with the greater centroid-Y (closer) takes priority:

```
# Both present — closest pillar wins
if best_green["centroid"][1] > best_red["centroid"][1]:
    # green is closer → steer left
else:
    # red is closer → steer right
```

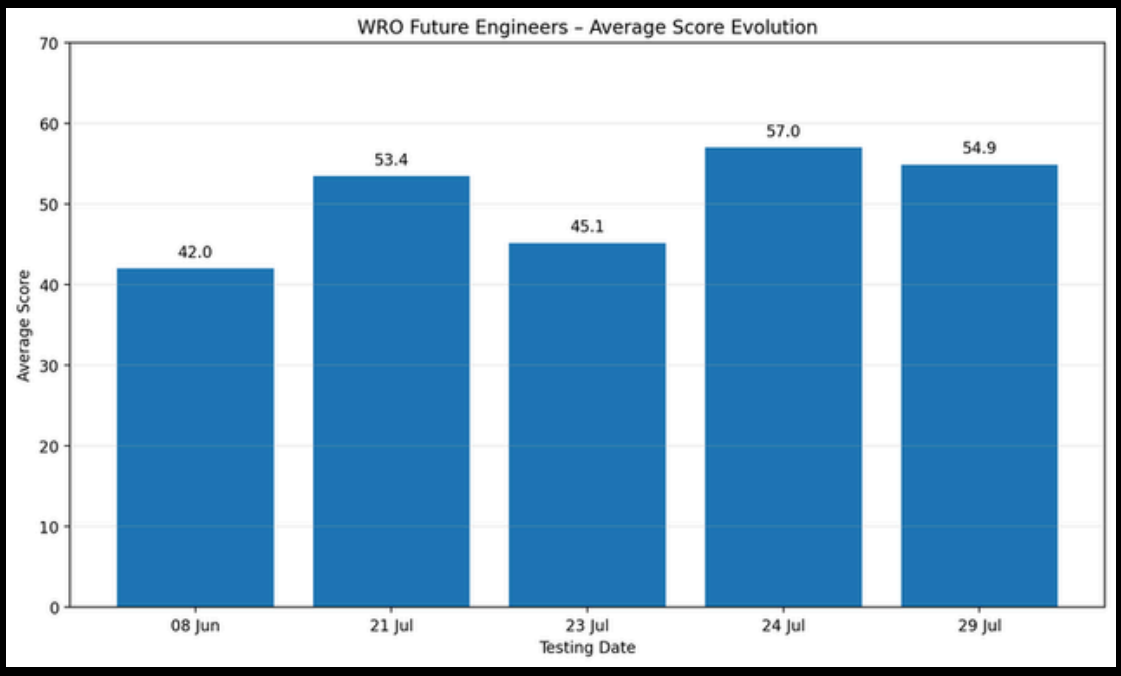
Camera settings fixed in code:

```
cap.set(cv2.CAP_PROP_EXPOSURE, -6)      # manual exposure, prevents auto-exposure flicker
cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)     # always process latest frame
```

Setting BUFFERSIZE = 1 ensures we always process the most recent frame. Without this, OpenCV buffers up to 3–4 frames internally, meaning at 30fps the detection could be 133ms stale - enough for the robot to travel past a pillar without reacting.
Post-run video reconstruction: During a run, frames and timestamps are written to disk. A separate build_video.py script reconstructs an annotated MP4 using ffmpeg's concat demuxer with per-frame durations - enabling post-run review without the overhead of cv2.VideoWriter in the live loop.

METRICE USED TO VALIDATE PERFORMANCE

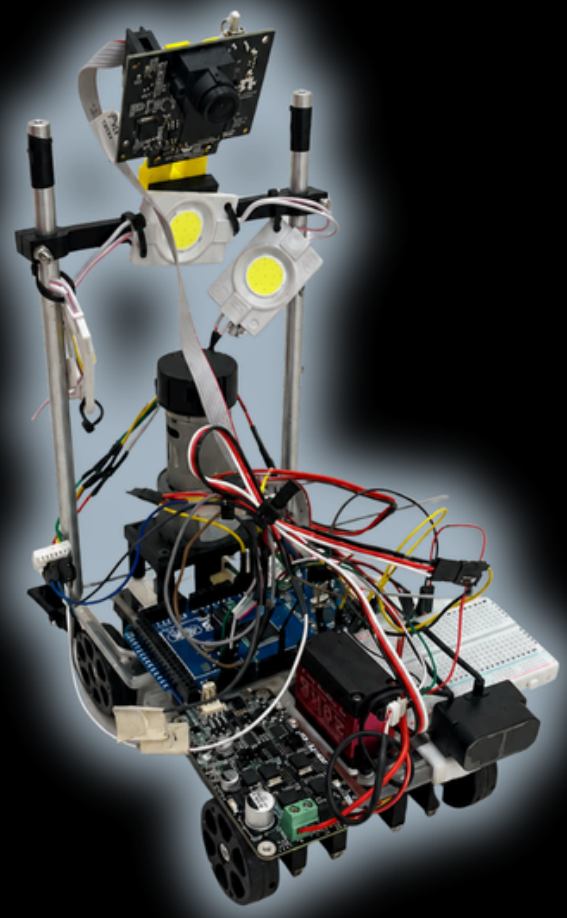
	Color	Points	Time	Issue	Resolved?
6/8/2026	Orange	29	0.23	detected red in ftc field (external)	Yes
6/8/2026	Orange	60	1.11	touched green block	Yes
6/8/2026	Orange	47	1.15	hit pink at start and end both	Yes
6/8/2026	Orange	47	1.15	hit pink in parking	Yes
6/8/2026	Blue	47	1.38	coding issue - wrong function callled	Yes
6/8/2026	Blue	47	1.28	didn't detect last wall roi	No
6/8/2026	Blue	47	1.21	moved forward after detecting last wall	No
6/8/2026	Blue	47	1.25	moved forward after detecting last wall	No
6/8/2026	Blue	47	1.09	moved forward after detecting last wall	No



SECTION 4

SYSTEMS THINKING AND ENGINEERING DECISIONS

V1



V2

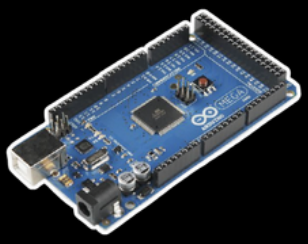
INITIAL ROBOT ELECTRONICS

Rhino GB37
1000RPM motor

Pixy2 60fps
1296·976px camera

BNO085 IMU

12V DC Coin LED
Lights (x2)



Arduino Mega
2560

GS2010MG 20kg · cm
Servo

MDD10A Dual-
Channel Motor
Driver

TF Mini Plus LiDAR
Distance Sensor

FINAL ROBOT ELECTRONICS

Vikram 453R6
Motor Driver

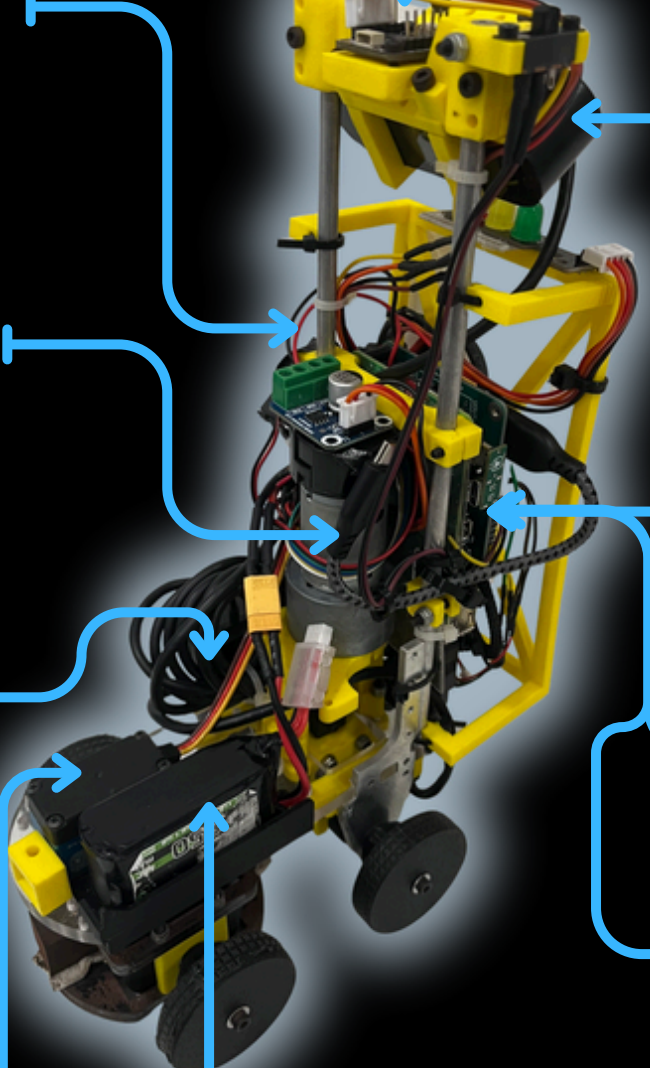
BNO085 IMU

Johnsons 900 RPM
12V DC Motor

HikVision USB
Webcam

Custom Designed
PCB

TF Mini Plus LiDAR
Distance Sensor



Raspberry Pi 4B

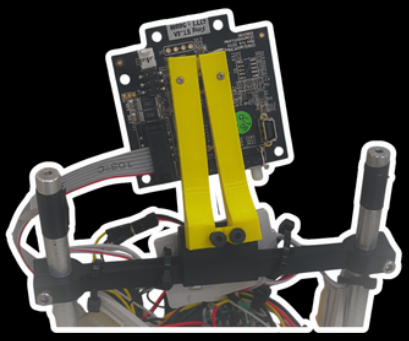
35kg-cm Servo

850mAh 11.1V
LiPo Battery

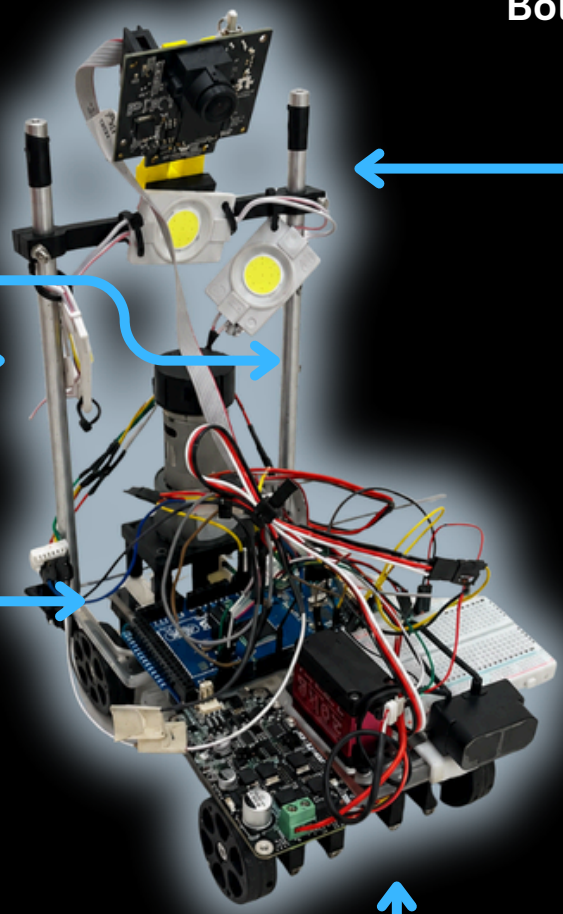
XIAO ESP32 C3

INITIAL ROBOT HARDWARE

Aluminium Camera Support Rods



Bolted Camera Mount

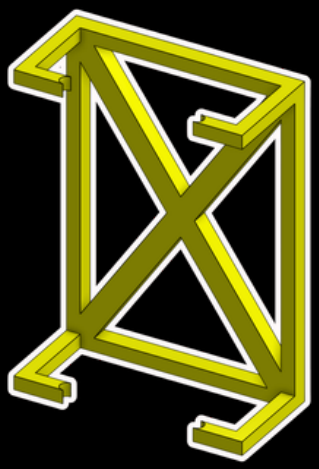


Lego (Plastic) Differential Gearbox

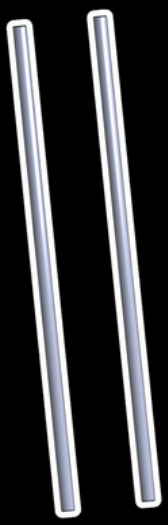
Parallel Steering System

FINAL ROBOT HARDWARE

3D printed clamped camera mount

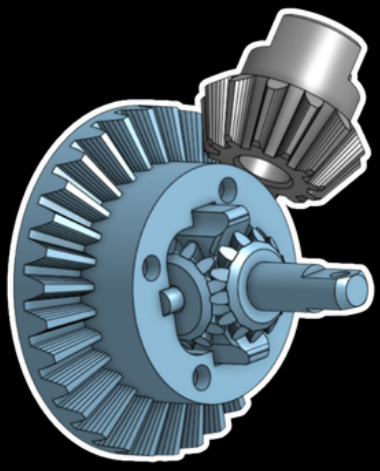


Electronics Cover

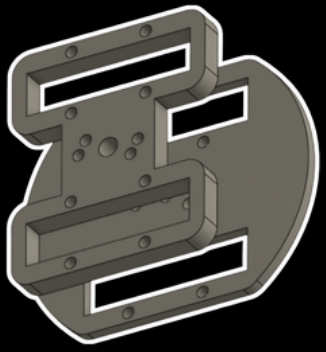


Aluminium Support Rods

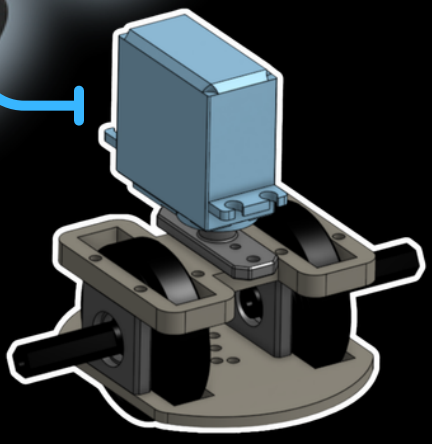
Rear Pivot Restrictor



1:3 Metal Differential Gearbox



Mild Steel Deadweight



Parallel servo steering

COMPARING ELECTRONICS: V1 VS FINAL

Primary Processor:

Arduino Mega 2560 (V1) vs Raspberry Pi 4B (V2)

- **Number of Cores:** We decided to switch to the Raspberry Pi 4B as its quad-core cortex allows it to run 4 processes at once, whereas the Arduno Mega can only run 1.
- **Hardware Specifications:** The Raspberry Pi also has superior RAM (4GB > 8KB), Storage (Raspberry Pi also external MicroSD card) and clock speed (1.5GHz > 16MHz)
- **Camera Compatibility:** The Raspberry Pi also allows us to use a USB webcam via USB 2.0 or USB 3.0. It also is compatible with OpenCV.

Secondary Processor:

None (V1) vs ESP32C3 XIAO (V2)

We decided to use a secondary processor (XIAO ESP32C3).

- **Reducing Electrical Noise:** Using a secondary processor for the IMU and encoder readings helped us reduce electrical noise (from the motor), stabilising their readings.

Motor Driver:

MDD10A (V1) vs Vikram 453R6 (V2)

- **Overcurrent Protection:** After the MDD10A faced 2 failures (due to not having overcurrent protection), we decided to switch to the Vikram 453R6 motor driver which adds overcurrent protection, thermal shutdown and auto-recovery.
- **Higher Voltage Ceiling:** The 453R6 can handle up to 45V DC input, making it capable of handling inductive spikes. This is much safer that the MDD10A's 30V DC ceiling.

COMPARING ELECTRONICS: V1 VS FINAL

DC Motor (Drive):

Rhino GB37 1000 RPM (V1) vs Johnsons 900 RPM (V2)

- **Torque:** With only 0.17 kgcm of torque, the Rhino GB37 (used in our version 1 robot) was inconsistent.
- We therefore switched to the Johnson's 900 RPM DC Motor with 1.2 kgcm rated torque.

Servo Motor (Steering):

GS2010MG Servo (V1) vs DS3235 Servo (V2)

- **Mitigating Risks:** The GS2010MG used in our first robot faced no issues, however noticing that the DS3235 has more torque, we switched to mitigate risks (especially considering that we planned to add a deadweight, making the parallel steering linkage heavier).

Main Camera:

Pixy2 Camera (V1) **vs** Hikvision USB DS-UO2 Camera (V2)

In our first robot version, the main reason we used the Pixy2 was because it was compatible with Arduino Mega. However, after switching to Raspberry Pi 4B, we changed our camera to the Hikvision USB DS-UO2 camera for its higher resolution and OpenCV capability.

Torch LEDs:

12V DC Coin LED (V1) **vs** None (V2)

At first, we tried to use these LEDs to allow our robot to work consistently in uneven ligthing. However, with such direct lighting, blocks were often confused with white. Therefore, we removed these LEDs.

COMPARING HARDWARE: V1 VS FINAL

Chassis Design:

Larger Chassis (V1) vs Slimmer Polycarbonate Chassis (V2)

- **Slimmer Chassis:** Our initial chassis had a wider layout, taking up unnecessary space. We redesigned it to be slimmer and more compact, reducing excess material and making the robot easier to manoeuvre around obstacles.
- **Weight Reduction:** The new design reduces unnecessary structural mass, lowering the load on our drive motor and allowing us to prioritise speed while maintaining stability.

Differential Gearbox

LEGO Plastic Differential (V1) vs 1:3 Metal Differential (V2)

- **Durability:** Our initial robot used a LEGO plastic differential gearbox. However, we switched to a metal differential for a more rigid and durable drivetrain that could withstand repeated high-speed runs.
- **Reliability:** The metal differential provides a stronger connection between the motor and rear wheels, while still allowing both wheels to rotate at different speeds during turns. This reduces tyre scrub and improves consistency.

COMPARING HARDWARE: V1 VS FINAL

Electronics Mounting

Basic Electronics Mounting (V1) vs Custom 3D-Printed Mounts (V2)

- **Space Optimisation:** Our initial electronics arrangement occupied more space and made component placement less organised. We designed custom 3D-printed mounts to secure the PCB and other electronics directly onto the aluminium support rods.
- **Reliability:** The new mounts keep components fixed during movement, reducing vibration and the risk of loose connections. This also makes wiring more organised and maintenance easier.

Camera Mounting

Bolted Camera Mount (V1) vs 3D-Printed Clamped Mount (V2)

- **Stability:** Our initial camera used a bolted mounting arrangement. We replaced it with a custom 3D-printed clamped mount that holds the camera securely on the support rods.
- **Design Flexibility:** 3D printing allowed us to design the mount around the camera's dimensions and intended position, giving us a more compact and integrated structure.

Mechanical Stability

Initial Chassis (V1) vs Deadweight and Rear Pivot Restrictor (V2)

- **Risk Mitigation:** Our final robot includes a front mild-steel deadweight and a rear pivot restrictor to reduce the risk of the robot tipping backwards, particularly during sudden acceleration or sharp turns.
- **Trade-Off:** These components increase the robot's mass, but we accepted this drawback to improve stability and protect the robot during runs.

IMU HEADING WRAPAROUND: EDGE CASE HANDLING

The problem:

- Heading is circular, so it jumps at the boundary - crossing from +179° to −179° is a 2° turn that naive subtraction reports as −358°.
- The PID cannot tell the difference - it sees a huge error and commands a full-lock turn in the wrong direction, usually mid-corner where recovery room is smallest.

The fix:

- Normalize the error, not the heading - wrapping the raw reading just moves the discontinuity; wrapping the difference removes it.
- Handle both directions - subtracting 360 above +180 fixes only half the circle; without the matching +360 branch below −180, the fault survives in one turn direction.

Pseudo Code:

```
if error_gyro > 180:
    error_gyro -= 360
elif error_gyro < -180:
    error_gyro += 360
```

HOW SUBSYSTEMS WORK TOGETHER

MOBILITY & SOFTWARE

Our drivetrain is controlled using PD steering and speed PID. Live sensor feedback allows the robot to continuously adjust its steering and speed instead of following fixed movements.

SENSORS & SOFTWARE

The camera and IMU work together to provide the software with vision, distance, and heading data. Combining multiple sensors makes navigation and obstacle avoidance more reliable.

POWER & SENSORS

We separated the motor and logic power rails to reduce electrical noise. This keeps sensor readings stable, even during rapid acceleration and steering.

FRAME & SENSORS

The chassis was designed around sensor placement. The elevated camera improves visibility, while the IMU is positioned to maximise accuracy and minimise interference.

FRAME & MOBILITY

Our lightweight, rigid chassis improves traction and stability while reducing load on the drivetrain. This allows faster, smoother, and more consistent movement.

OVERALL SYSTEM

Each subsystem supports the others. The frame improves sensor performance, sensors guide the software, the software controls mobility, and the power system keeps everything running reliably, creating a robot that can adapt dynamically without excessive hard-coding.

RISK IDENTIFICATION / MITIGATION

We identified the following risks and mitigated them.

Risk Identified	Mitigation
Robot tipping during high acceleration.	Added dead weight to lower the centre of gravity and widened the wheelbase to improve stability and prevent tipping.
Wiring failures	Designed and manufactured a custom PCB to make the wiring more secure, organized, and reliable.
Limited space for electronics	Replaced the MDD3A motor driver with the more compact 7Semi Vikram-453R6, freeing up space for other components.

ENGINEERING DECISIONS...

PYTHON THREADING V/S MULTIPROCESSING

Problem

- One loop, everything in it - camera, HSV, IMU, encoder, steering, in sequence.
- Blocking reads cost 10–15 ms/cycle, capping the loop at ~65–100 Hz before any logic ran.
- Sense-to-steer lag left the robot committing to stale data, and a busy frame stretched it further mid-run.

Solution

- Multiprocessing, not threading - the GIL serialises CPU-bound work like OpenCV, so threads buy no real parallelism.
- Three processes across four Pi 4B cores - vision and sensor write to shared memory, drive reads only and never blocks.
- Loop rate is now decoupled from frame arrival - a slow frame reuses the last vision result instead of stalling steering.

Trade-off

- Not value corruption - multiprocessing.Value already locks single accesses.
- Inconsistency across variables - head.value and dist.value read separately, updated in between, giving a state the robot was never in.
- Fix: one lock, copy all values locally, release, then compute - at the cost of minor contention and harder cross-process debugging.

ENGINEERING DECISIONS

CENTROID PD V/S ENCODER DEAD RECKONING

Old approach:

- Dead-reckoning position estimate - an EncoderCounter maintained an (x, y) estimate within each straight section.
- PD on positional error - correctPosition drove estimated position toward a setPoint lateral offset.
- Avoidance shifted the setpoint - a detected pillar biased setPoint to ±35 (right/left).

Why we moved away?

- Error accumulated across sections, so the estimate was only as good as the last reset.
- The wall-reset fix added its own failure modes - TOF reset logic became a second thing that could go wrong.
- Drift of ±30 units by lap 3 was still causing avoidance misses at speed.

This season's approach:

- Vision drives avoidance directly - err is a pixel offset from a target centroid X (e.g. 350 - centr_x for green in state 1).
- PD on pixel error - kp_s and kd_s from gains.json output a servo angle correction.
- No positional state, so no drift - it runs at camera FPS, and setpoints are tunable straight from video review.

ENGINEERING DECISIONS...

SINGLE V/S MULTI-STAGE PARKING

Challenge Expectations:

Since the WRO 2026 rules expect the robot to be fully inside the parking lot, and parallel to the outer wall, a simple forward drive into the slot fails, parking the robot at an angle.

Our Solution:

We developed a multi-stage (4 step) parking sequence to park our bot inside the parking slot, parallel to the wall.

1. Reverse 100 encoder counts straight (exit the approach angle)
2. Steer heading ±90° and reverse into the slot until corr < 10°, then reverse 1.5s more for depth
3. Short forward burst (0.55s): first 0.2s straight, then servo to 70°/110° to press into the slot
4. Final heading correction (heading - 90°), reverse until straight (corr < 3°), then drive forward until centr_y_pink > 235 (pink border fills the bottom of frame = robot fully inside)

Following these 4 steps allows our bot to park within the slot consistently, being parallel to the wall.

Trade-Offs Acknowledged:

Although this parking sequence takes more time than simply driving into the parking slot, it ensures accuracy. Also, since our bot moves quickly, our robot makes finishes the course inside the time limit even with the extended parking sequence.

ENGINEERING DECISIONS...

ARDUINO MEGA VS RASPBERRY PI 4B

Our First Solution:

- Arduino Mega: Previously used as primary processor for all processes.
- Only has 1 core, forcing all processes to be run in one loop (leading to very high loop time).
- Low RAM, storage and clock speed further increasing loop times.
- Not compatible with OpenCV, limiting camera options to ones with built-in colour recognition software (such as Pixy). Also not compatible with USB webcams.

Our New Solution:

- Replaced Arduino Mega with Raspberry Pi 4B.
- Quad-core processor allows multiple separate processes to run at once (multiprocessing).
- Higher RAM and clock speed. Supports external microSD storage card.
- Has USB webcam compatibility (via USB 2.0 or 3.0). Compatible with OpenCV. This allowed us to use HikVision Webcam.

Feature	Arduino Mega	Raspberry Pi 4B
Core Processor	Single Core ATmega 2560	BCM2711 Quad-core Cortex
Clock Speed	16 MHz	1.5 GHz
RAM	8 KB	4 GB
Storage	256 KB Flash Memory	Compatible with external microSD card.

Adding a Secondary Processor:

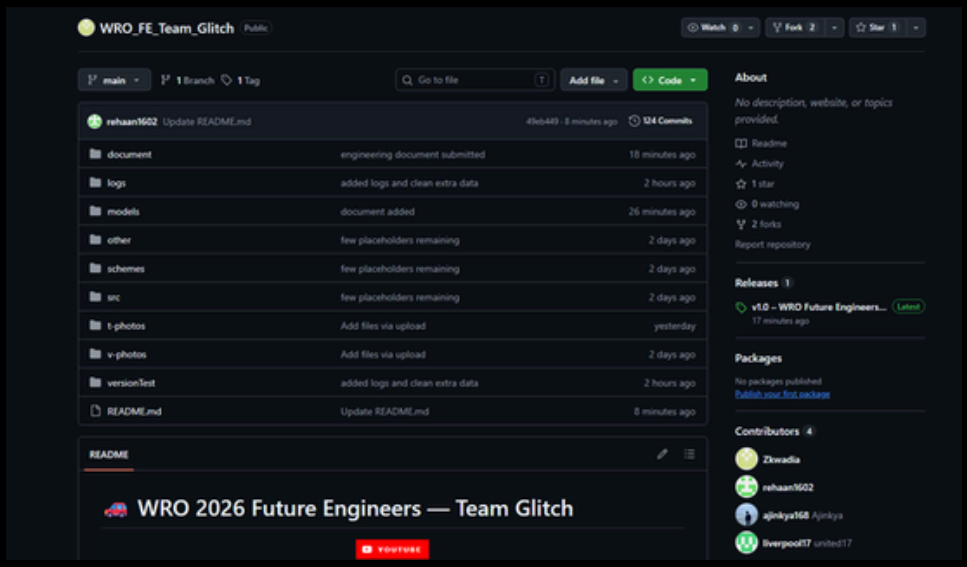
- We added an ESP32C3 XIAO to read encoder and IMU inputs.
- This allowed us to keep these sensors on a separate rail from the motor, making their readings more reliable and consistent.

SECTION 5

REPRODUCIBILITY AND GIT QUALITY

GIT REPOSITORY

Reproducibility promise : Everything needed to rebuild this vehicle from raw materials and run it on a competition field is in this repository: CAD source and STL/DXF exports, the full bill of materials with part numbers, wiring diagrams, firmware, control software, calibration procedures, and the test protocol we used to validate every subsystem. A team with no prior contact with us should be able to reproduce our vehicle. If any step in this document is ambiguous, that is a documentation bug - please open an issue.



Git Link : https://github.com/Zkwadia/WRO_FE_Team_Glitch/

Code Structure

```
WRO_2026/
├── versionTest/
│   ├── obstacle_3.py              # Obstacle - 4-process
│   ├── open_2.py          # Open - 4-process
│   ├── vision_pipeline_new.py     # HSV pipeline - obstacle
│   ├── vision_pipeline_3.py       # HSV pipeline - open
│   ├── TFmini.py                  # TFmini Plus UART reader
│   ├── Encoder.py                 # Dead-reckoning (open challenge)
│   ├── Servo.py                   # Servo abstraction (pigpio)
│   ├── build_video.py             # Post-run MP4 reconstruction
│   └── gains.json                 # Hot-reloadable PID gains
├── src/xiao/imu_encoder.ino       # XIAO firmware
├── models/
│   ├── 3D prints/
│   └── STEP/
├── schemes/wiring_diagram.png
└── v-photos/
```

Built for rebuilding, not reading: editable CAD with STL/DXF exports, sourced BOM, wiring schematic, power budget, firmware, eight-stage build guide, and a hardware-substitution table mapping swaps to config keys and tests.

STEP 1 - 3D PRINT THE CHASSIS

All the CAD and structural files are attached in the models folder of our Github Repository, in the structure shown below.

```
odels/
— 3D prints/                              # STL and project files — slice and print directly
   ├— all part 3d print file.3mf          # Complete multi-part print project (open in Bambu/PrusaSlicer)
   ├— project file.3mf                    # Alternate slicer project
   ├— new camera mount.STL               # Camera mount with 10° downward tilt
   ├— long stick.STL                     # Vertical sensor mast
   ├— long stick mount.STL               # Base mount for sensor mast
   ├— front cover.STL                    # Front bumper cover
   ├— battery 1.STL                      # Battery holder
   ├— an_6_12_holder.STL                 # 6mm/12mm rod holder
   ├— an_spacer internationale.STL       # Spacer for international chassis spec
   ├— spacer with tf mount.STL           # Spacer with sensor bracket
   ├— tf mini mount horizontal.STL       # Horizontal sensor bracket (legacy, for reference)
   ├— atfmini mounting.STL               # Alternate sensor mounting bracket
   ├— rp lidar mount.STL                 # RPLidar mount (legacy, for reference)
   └— an waveshare 5 inch dsi lcd.STL    # Display mount (optional)
— STEP/                                  # Editable CAD files (SolidWorks, Fusion 360, OnShape, FreeCAD)
   ├— WRO FE Car complete CAD.STEP       # Full robot assembly — start here
   ├— chassis_Polycarbonate_6mm thickness_qty1pc.STEP   # Chassis plate (cut from 6mm polycarbonate)
   └— electronics Mount_3mm polycarbonate_ qty 1.STEP   # Electronics tray (cut from 3mm polycarbonate)
```

Important Notes:

The chassis and electronics tray are laser-cut or CNC-cut from polycarbonate sheets. All files in the 3D prints folder are printed parts. The .3mf files contain pre-arranged plates with all parts oriented.

Recommended Print Settings:

Parameter	Written By
Material	PETG or PLA+ (PETG prefferred)
Layer Height	0.2 mm
Infill	40% gyroid
Perimeters / Walls	4
Supports	Required for “new camera mount.STL” and “long stick mount.STL” only
Bed Adhesion	Brim for any part with a narrow base

Tip for Assembly:

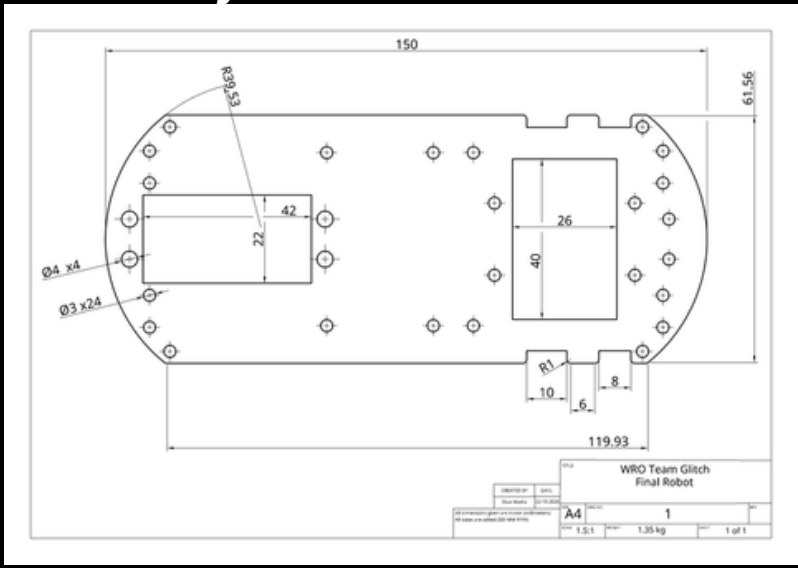
Open WRO FE Car complete CAD.STEP in your CAD tool to see how every part fits together before printing or cutting anything.

STEP 2 - LASER CUTTING

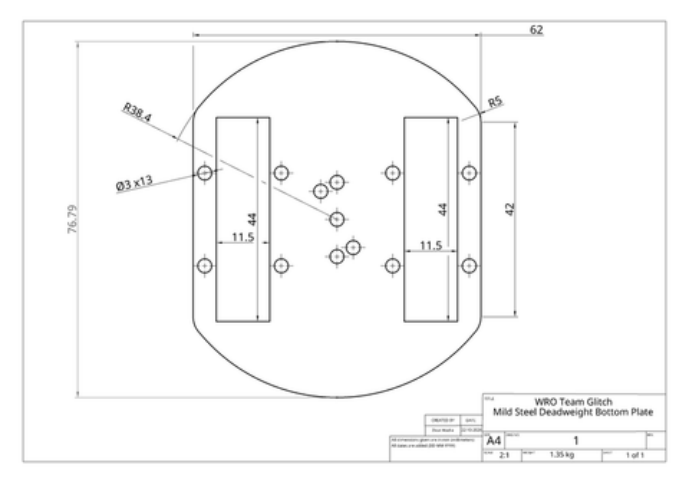
- Refer to ‘WRO FE Car complete CAD.STEP’ as it shows all of the components in their final, installed position.

- For laser-cutting the polycarbonate chassis, and deadweight, .STEP files are provided. Refer to the following drawings.

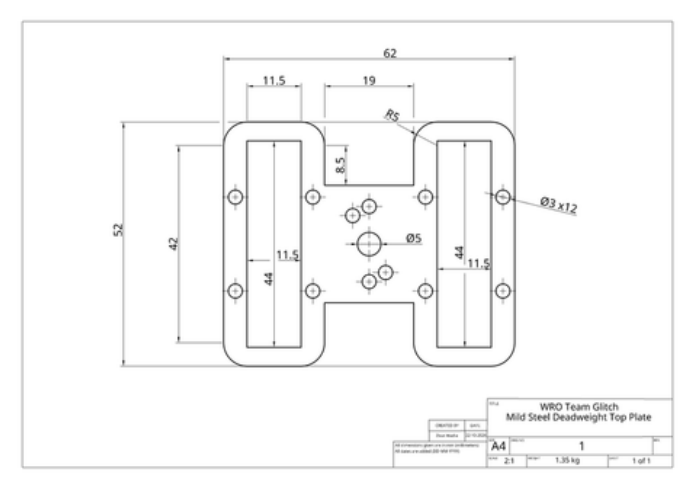
Polycarbonate Chassis



Deadweight (bottom piece)



Deadweight (top piece)



- Cut the polycarbonate chassis (6mm polycarbonate chassis.dxf) out of 6mm polycarbonate. Cut the electronics tray (electronics Mount_3mm polycarbonate_ qty 1.STEP) out of 3mm polycarbonate. Cut the deadweights (3mm mild steel deadweight upper.dxf **AND** 3mm mild steel deadweight lower.dxf) out of 3mm mild steel.

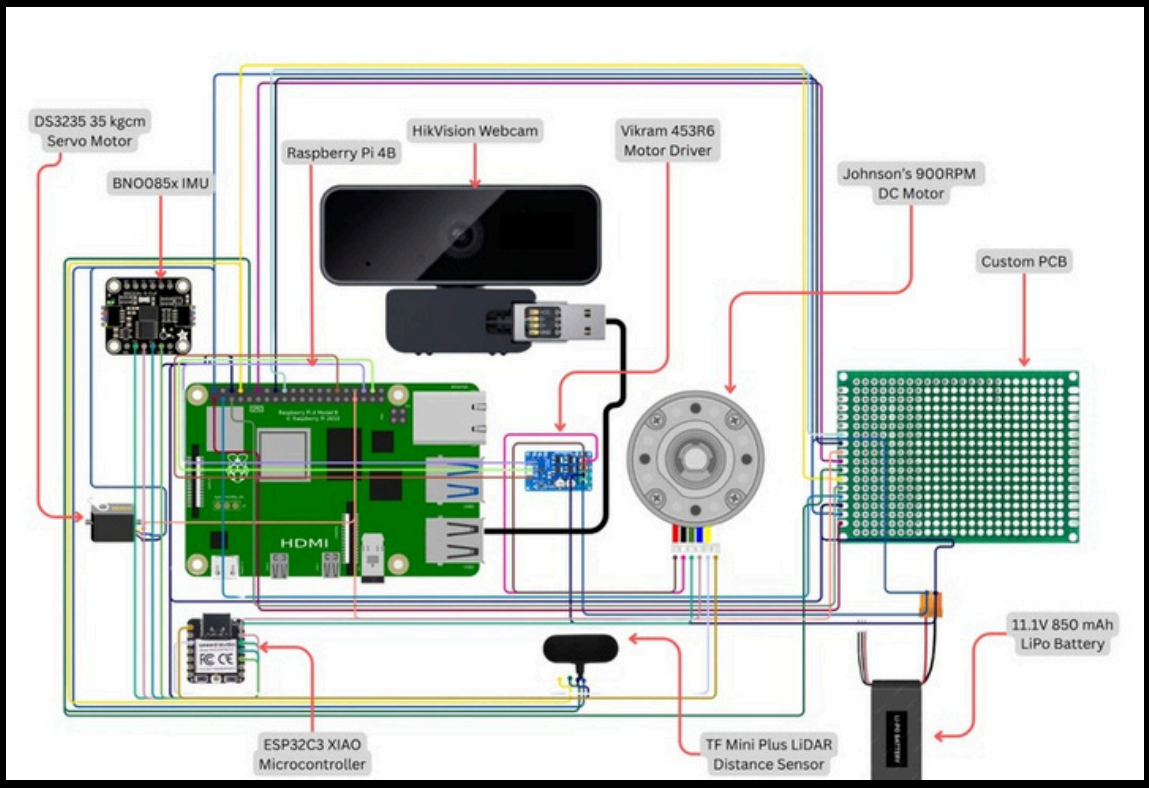
STEP 3 - MECHANICAL ASSEMBLY

- Drive motor: Mount the Johnsons 900RPM DC motor into the rear motor bay using M3 bolts. Route the encoder cable forward along the chassis underside.
- Steering servo: Press the DS 35kg·cm servo into the front servo bay and connect to the Ackermann steering linkage.
- Front axle: Install the Ackermann steering linkage. Both front wheels should turn at slightly different angles at full lock - verify the inner wheel turns sharper than the outer wheel.
- Sensor mast: Print long stick.STL and long stick mount.STL. Mount the mast vertically at the front-centre of the chassis.
- Camera mount: Print new camera mount.STL and attach to the top of the sensor mast or front rail. The mount has a fixed 10° downward tilt - no adjustment needed.
- Front cover: Print front cover.STL and clip onto the front bumper rail to protect sensor wiring.
- Battery holder: Print battery 1.STL and secure the LiPo pack to the chassis deck.
- Electronics tray: Screw the cut 3mm polycarbonate tray to the chassis deck. Mount the Raspberry Pi, XIAO ESP32-C3, and MDD3A onto the tray using the pre-drilled mounting bosses visible in the STEP file.

STEP 4 - ELECTRONICS WIRING

Wire all components according to the wiring diagram in schemes/wiring_diagram.png.

Below is the diagram for the power rails.



Critical wiring rules:

- The BNO085 must be powered from the XIAO's 3V3 rail, not the Pi's rail. Sharing the Pi rail introduces motor EMI into the IMU heading.
- Run the motor wires and encoder wires on opposite sides of the chassis - motor current induces EMI in nearby encoder signal lines.

STEP 5 - FLASH THE XIAO ESP32-C3 FIRMWARE

1. Install *Arduino IDE (≥2.0)* and add the Seeed Studio board package:
    - In Arduino IDE: File → Preferences → Additional boards manager URLs
    - Add: https://files.seeedstudio.com/arduino/package_seeeduino_boards_index.json
    - Tools → Board → Boards Manager → search "Seeed XIAO ESP32C3" → Install
2. Install required libraries via Library Manager:
    - Adafruit BNO08x
    - Adafruit Unified Sensor
3. Open src/xiao/imu_encoder.ino, select board XIAO ESP32C3, select the correct COM/tty port, and upload.

The firmware reads BNO085 Euler heading over I2C and quadrature encoder counts via GPIO, then broadcasts over USB UART at 115200 baud in the format: <heading_float> <encoder_int>\n

Serial commands accepted at runtime:

- b"R" - software reset (reboots XIAO and reinitialises BNO085)
- b"1" - zero the encoder count

STEP 6 - SET UP THE RASPBERRY

OS: Raspberry Pi OS 64-bit Bookworm (Lite or Desktop). Enable I2C and the camera interface via raspi-config.

- Install system dependencies:
  - *sudo apt update && sudo apt install -y python3-pip pigpio python3-pigpio ffmpeg i2c-tools*

Install Python packages:
pip3 install opencv-python pyserial RPi.GPIO adafruit-circuitpython-vl53l0x numpy

Verify I2C bus and sensors:

- # Start pigpio daemon
  - *sudo pigpiod*

Set up USB device alias so /dev/XIAO_USB always resolves to the XIAO (plug the XIAO in first):

- # Find vendor and product ID
  - *lsusb # note idVendor and idProduct for the XIAO# Create udev rule*
- *sudo nano /etc/udev/rules.d/99-wro.rules*
- *Add this line (replace XXXX/YYYY with your VID/PID from lsusb):*
- *SUBSYSTEM=="tty", ATTRS{idVendor}=="XXXX", ATTRS{idProduct}=="YYYY", SYMLINK+="XIAO_USB"*
- *sudo udevadm control --reload-rules && sudo udevadm trigger*

Clone the repository:

- *cd /home/pi*
- *git clone https://github.com/YOUR_TEAM/WRO_2026.git*
- *cd WRO_2026/versionTest*

STEP 7 - CONFIGURE PID GAINS

Edit gains.json before the first run. Start conservative and tune iteratively - the file hot-reloads while the script is running so you do not need to restart between adjustments:

```
{
  "kp": 0.6,
  "kd": 0.01,
  "ki": 0,
  "kp_s": 0.5,
  "kd_s": 0.5,
  "kp_v": 0.01,
  "ki_v": 0,
  "kd_v": 0,
  "SPEED_SCALE": 2447.0
}
```

kp/kd/ki - heading PID (servo). *kp_s/kd_s* - centroid/wall PD (servo). *kp_v/ki_v/kd_v* - speed PID (motor duty, obstacle challenge only). SPEED_SCALE - encoder counts per second at 100% duty on a fresh battery; calibrate this from your first run logs before enabling ki_v or kd_v.

STEP 8 - RUN

Open Challenge: This is our path, either copy our repo or use your own
cd /home/pi/WRO_2026/versionTest
sudo python3 open_challenge.py

Obstacle Challenge: This is our path, either copy our repo or use your own
sudo python3 obstacle_3.py

Both scripts start pigpiod automatically. Logs are written to /home/pi/wro_logs/logs/ with timestamps. Flip the kill switch (GPIO 12) to start/stop the robot without restarting the script.